



同济大学
TONGJI UNIVERSITY



土木工程学院
COLLEGE OF CIVIL ENGINEERING

第五章 ROS2仿真

机器人仿真

潘玥

2025年5月5日



■ ROS 2 核心要素

- 工作空间 (Workspace) : 项目的文件系统, 组织源码、编译脚本及生成文件
- 功能包 (Package) : 功能模块单元, 组织一类特定功能的节点
- 节点 (Nodes) : 独立运行的程序模块, 执行特定任务

■ ROS 2 通信机制

- 话题 (Topics) : 节点间异步通信, 发布/订阅模型
- 服务 (Services) : 同步请求/响应通信, 适合一次性任务
- 动作 (Actions) : 异步、带反馈的长时间任务

■ ROS 2 的能力

- 提供通信框架, 连接机器人硬件、传感器和算法
- 支持模块化开发, 降低机器人控制的复杂性



■ 第一阶段任务：简易雷视系统

- 熟悉嵌入式开发环境，完成软硬件集成
- 掌握传感器、舵机通信与控制方法



■ 第二阶段任务：简易雷视系统+移动底盘

- 结合现有硬件条件，选择土木工程场景，实现底盘遥控下的巡检、测量等，形成报告。
- 基于仿真实现机器人更复杂的功能和应用



■ 第三阶段任务：自主导航机器人

- 基于仿真环境实现机器人目标搜索任务
- *硬件实现



■ 期末任务：虚拟仿真机器人抓取

- 基于仿真环境实现目标搜索并完成物体搬运任务
- *硬件实现



目录

CONTENTS

1

机器人坐标变换

2

机器人仿真方法

3

机器人建模语言

4

机器人建模案例



机器人坐标变换



同济大学
TONGJI UNIVERSITY



土木工程学院
COLLEGE OF CIVIL ENGINEERING



3.1 机器人坐标变换需求

■ 问题：机器人上通常有许多传感器用于感知周边物体方位

- 获得的点云数据，对于机器人而言是哪个位置？
- 机器人离目标点有多远？

■ 这些数据通常以坐标、横/纵/高的距离信息呈现，表征了物体和传感器之间的方位关系。



毫米波雷达



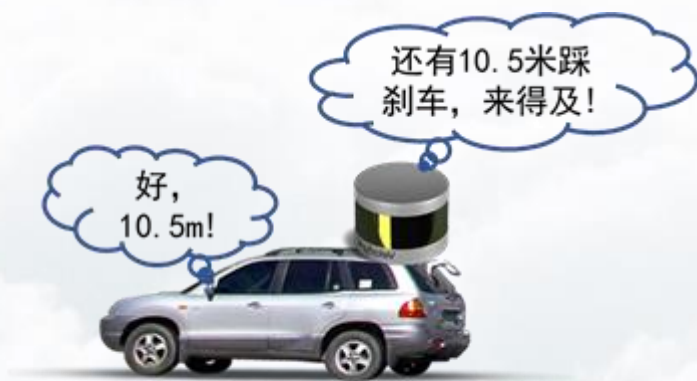
深度相机



激光雷达

■ 需求：如何运用这些信息进行机器人控制？

- 数据测量的需要：需要把传感器采集的数据映射到世界坐标系
- 机器人控制的需要：需要根据感知信息计算出各种关节的运动目标点





概念：自由度

工业机器人是由一系列连杆通过铰链（关节）顺序连接而成的操作臂， 机器人机构的基本元素为连杆和铰链（关节）（又称**运动副**）

自由度（DOF）：

机构的自由度是指确定机构位姿所需独立参数的数目

DOF=6： **满自由度机构**

DOF>6： **冗余自由度机构**

DOF<6： **欠自由度机构**

自由度计算CGK公式：
$$F = d(l - n - 1) + \sum_{i=1}^n f_i$$

其中：

d ：为机构阶数（对于平面机构 $d=3$ 、对于空间机构 $d=6$ ）

l ：连杆数（包括基座）

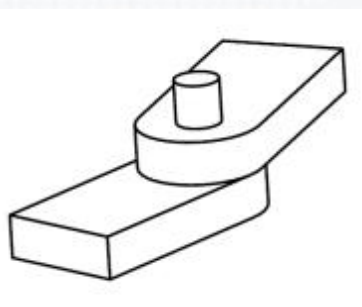
n ：关节总数

f_i ：第 i 个关节的自由度数

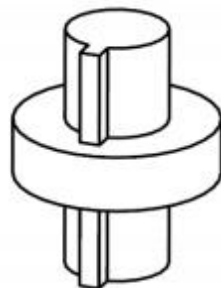
理解：位置+姿态



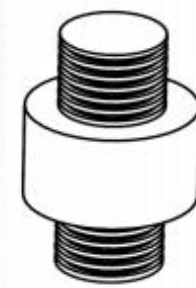
概念： 自由度



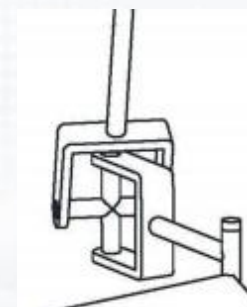
旋转副 (R)
1个自由度



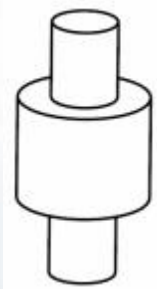
移动副 (P)
1个自由度



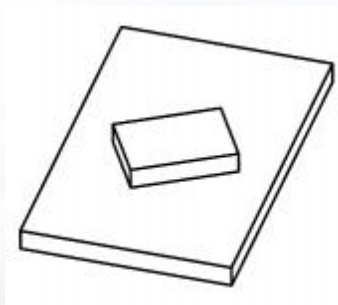
螺旋副 V (H)
1个自由度



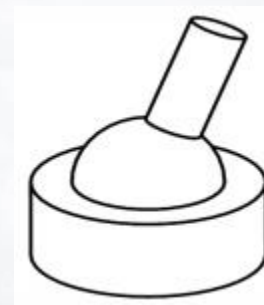
虎克铰 (U)
2个自由度



圆柱副 (C)
2个自由度



平面副 (E)
3个自由度

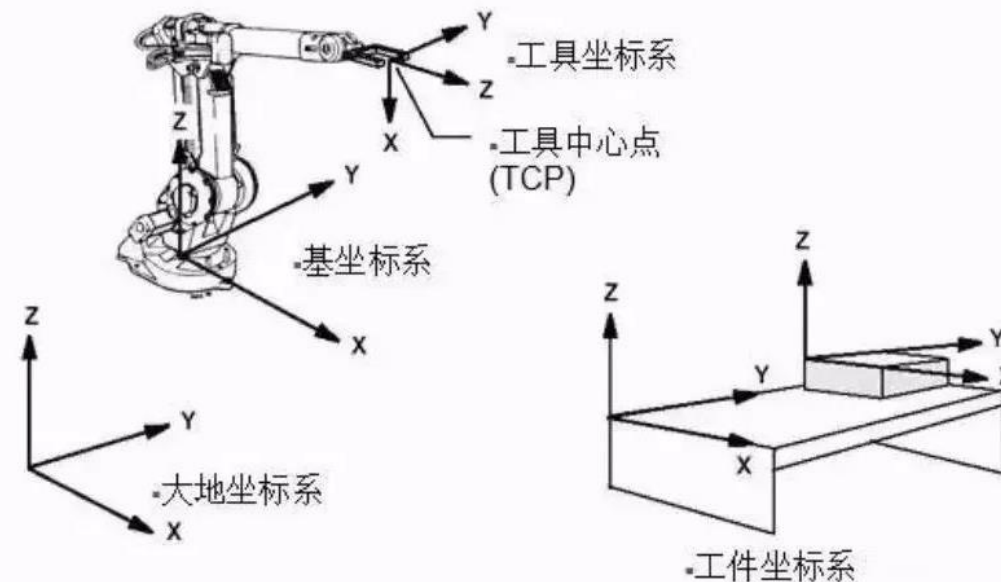
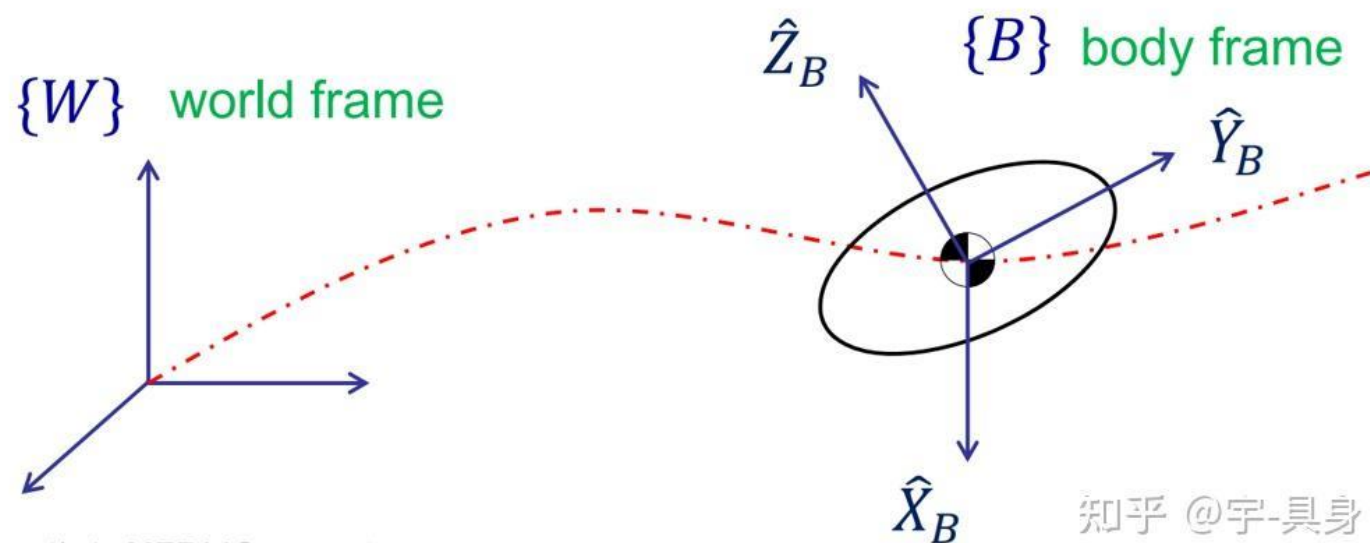


球面副 (S)
3个自由度



3.2 机器人的坐标系 (Frame)

- 世界坐标系：以地面为基准的三维笛卡尔直角坐标系，可用来描述物体相对于地面的运动。
- 机器人坐标系：可用来描述机器人本体运动的虚拟笛卡尔直角坐标系。



每一个运动副对应一个坐标系
机器人坐标系均遵守右手法则



3.2 三维空间刚体运动

■ 位姿 = 位置 (点) + 姿态 (方位)

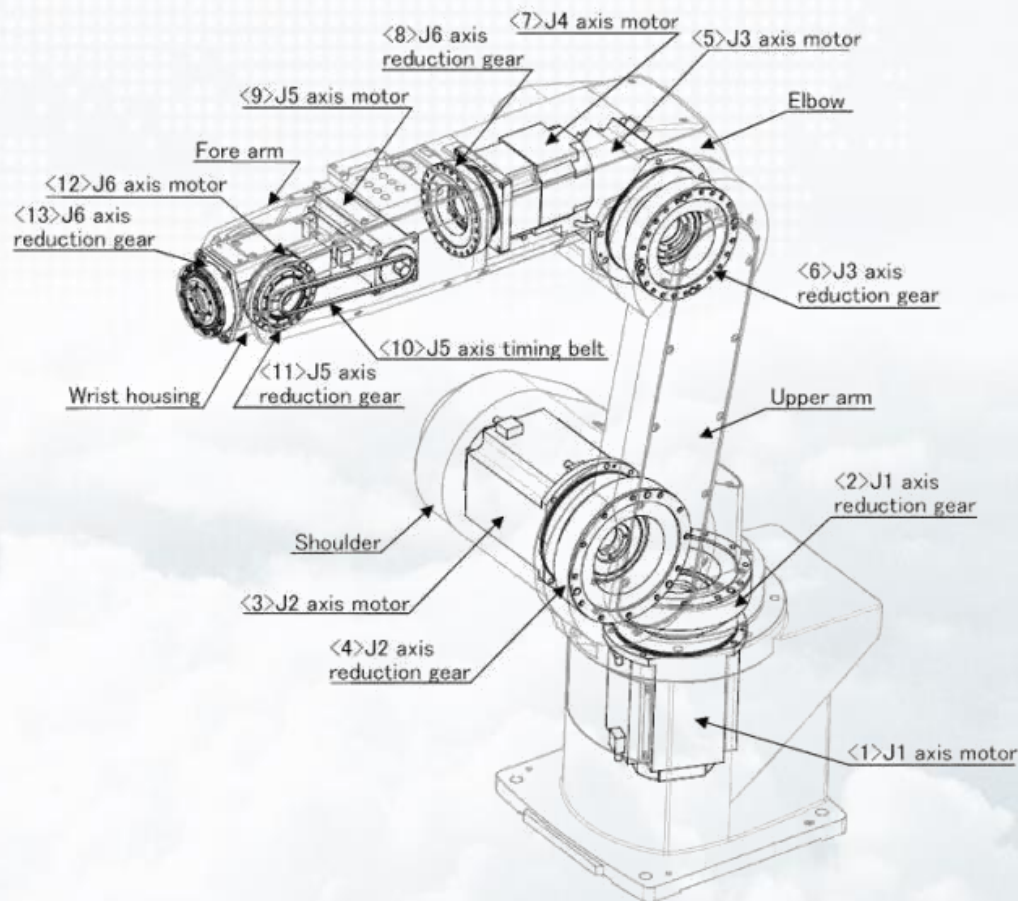
位置: 描述刚体在某一参考坐标系中的位置

姿态: 描述刚体的朝向或旋转状态

■ 位姿变换

➤ 平移

➤ 旋转



■ 机器人基座坐标系中, 如何描述末端执行器的位姿?



3.2 三维空间刚体运动

■ 位置描述：笛卡尔坐标系

坐标系{A}中，空间 p 点可表示为列向量 ${}^A\mathbf{p}$ ：

坐标系{B}中，空间 p 点可表示为列向量 ${}^B\mathbf{p}$ ：

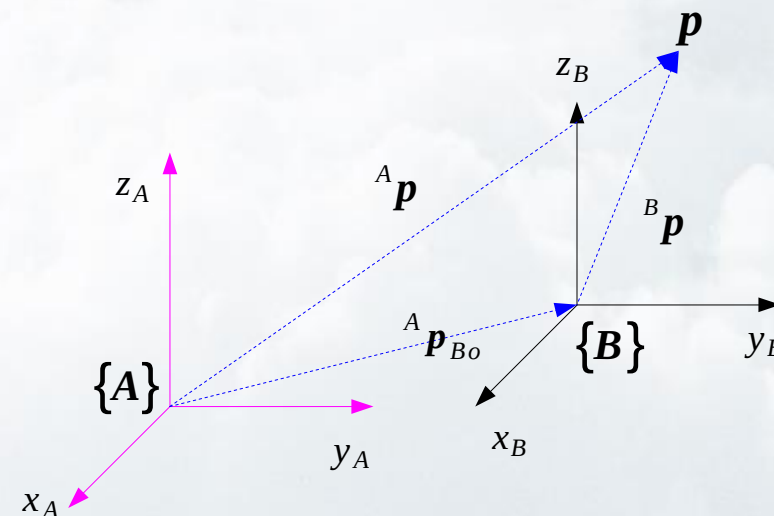
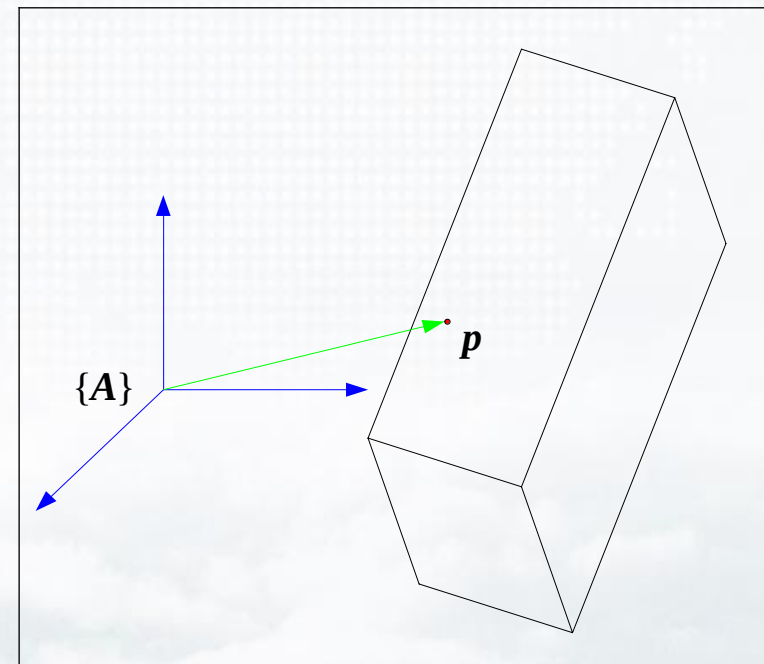
$${}^A\mathbf{p} = \begin{bmatrix} {}^A p_x \\ {}^A p_y \\ {}^A p_z \end{bmatrix}$$

$${}^B\mathbf{p} = \begin{bmatrix} {}^B p_x \\ {}^B p_y \\ {}^B p_z \end{bmatrix}$$

■ 平移变换

$${}^A\mathbf{p} = {}^B\mathbf{p} + {}^A\mathbf{p}_{B0}$$

${}^A\mathbf{p}_{B0}$ ：B坐标系原点在A坐标系中的坐标





3.2 三维空间刚体运动

■ 位置描述：笛卡尔坐标系

坐标系{A}中，空间 p 点可表示为列向量 ${}^A\mathbf{p}$ ：

坐标系{B}中，空间 p 点可表示为列向量 ${}^B\mathbf{p}$ ：

$${}^A\mathbf{p} = \begin{bmatrix} {}^Ap_x \\ {}^Ap_y \\ {}^Ap_z \end{bmatrix} \quad {}^B\mathbf{p} = \begin{bmatrix} {}^Bp_x \\ {}^Bp_y \\ {}^Bp_z \end{bmatrix}$$

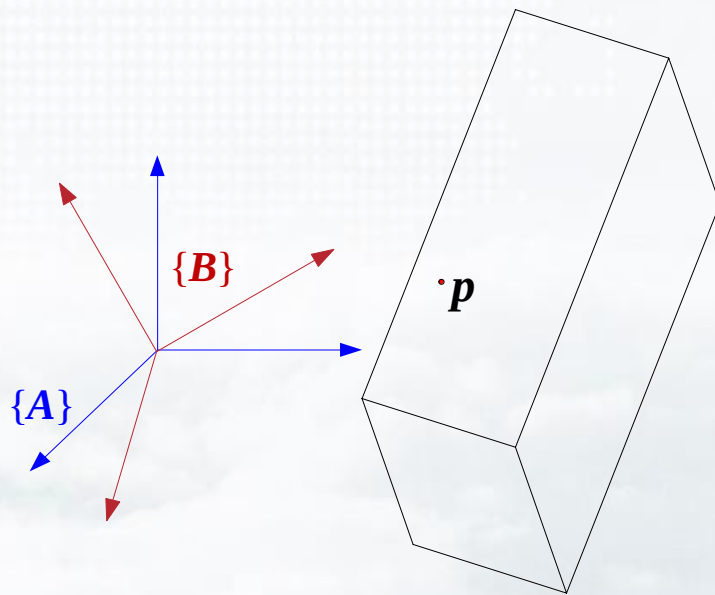
■ 旋转：若空间 p 点不动，坐标系{A}旋转为坐标系{B}：

$$\begin{bmatrix} {}^Ae_x & {}^Ae_y & {}^Ae_z \end{bmatrix} {}^A\mathbf{p} = \begin{bmatrix} {}^Be_x & {}^Be_y & {}^Be_z \end{bmatrix} {}^B\mathbf{p}$$

$${}^A\mathbf{p} = \begin{bmatrix} {}^Ae_x & {}^Ae_y & {}^Ae_z \end{bmatrix}^T \begin{bmatrix} {}^Be_x & {}^Be_y & {}^Be_z \end{bmatrix} {}^B\mathbf{p}$$

$${}^A\mathbf{p} = \begin{bmatrix} {}^Ae_x^T {}^Be_x & {}^Ae_x^T {}^Be_y & {}^Ae_x^T {}^Be_z \\ {}^Ae_y^T {}^Be_x & {}^Ae_y^T {}^Be_y & {}^Ae_y^T {}^Be_z \\ {}^Ae_z^T {}^Be_x & {}^Ae_z^T {}^Be_y & {}^Ae_z^T {}^Be_z \end{bmatrix} {}^B\mathbf{p}$$

$${}^A\mathbf{p} \triangleq {}^A_B\mathbf{R} {}^B\mathbf{p}$$



■ ${}^A_B\mathbf{R}$ 为旋转矩阵

■ 正交矩阵

■ 行列式为1

■ ${}^A_B\mathbf{R} = {}^B_A\mathbf{R}^{-1} = {}^B_A\mathbf{R}^T$

■ ${}^A\mathbf{p} \triangleq {}^A_B\mathbf{R} {}^B\mathbf{p}$ 表示{B}到{A}的旋转

■ ${}^B\mathbf{p} \triangleq {}^B_A\mathbf{R} {}^A\mathbf{p}$ 表示{A}到{B}的旋转

3.2 三维空间刚体运动

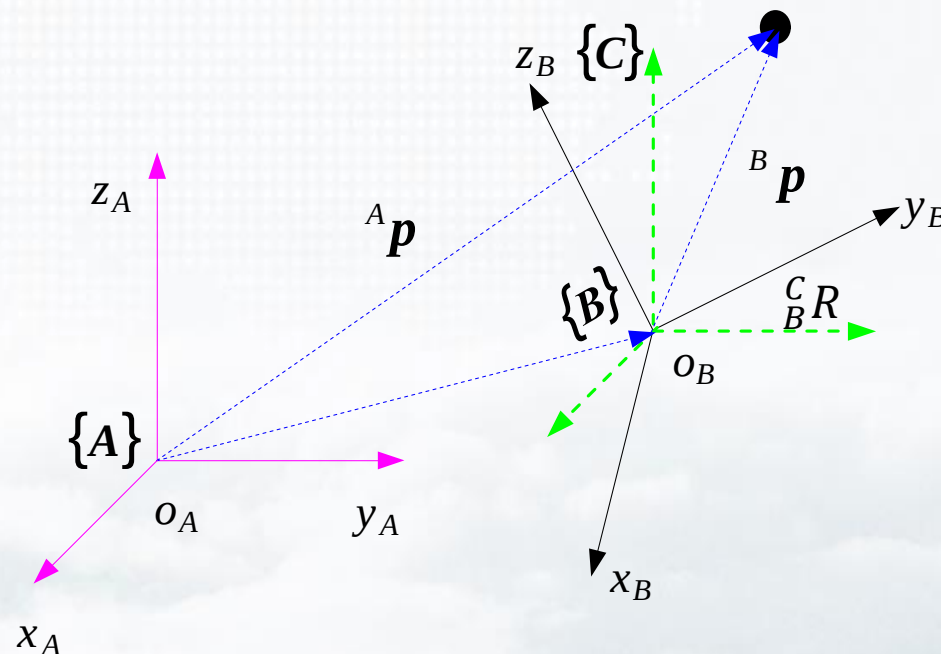
■ 一般变换：两个坐标系间的运动可用旋转和平移完全描述

$${}^C p = {}^C R {}^B p = {}^A R {}^B p$$

$${}^A p = {}^A R {}^B p_B + {}^A p_{B0}$$

{B}相对于{A}
的旋转矩阵

{B}的原点相对于
{A}的平移向量

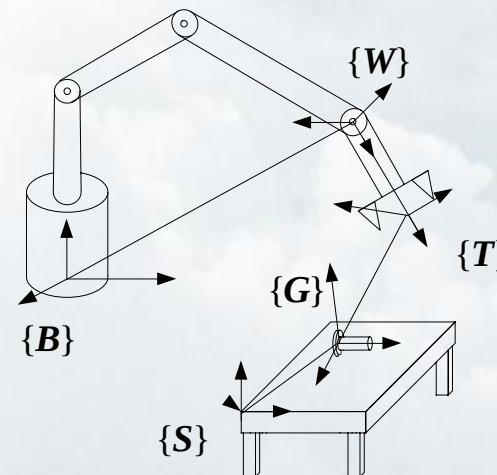


■ 齐次变换：考虑级联坐标变换中有不便之处

$$b = R_1 a + t_1, \quad c = R_2 b + t_2. \quad \longrightarrow \quad c = R_2 (R_1 a + t_1) + t_2.$$

$${}^A p = {}^A R {}^B p + {}^A p_{B0} \quad \longrightarrow \quad \begin{bmatrix} {}^A p \\ 1 \end{bmatrix} = \begin{bmatrix} {}^A R & {}^A p_{B0} \\ 0 & 1 \end{bmatrix} \begin{bmatrix} {}^B p \\ 1 \end{bmatrix} \quad \longrightarrow \quad \begin{bmatrix} {}^A p \\ 1 \end{bmatrix} = {}^A T_B \begin{bmatrix} {}^B p \\ 1 \end{bmatrix}$$

$$\tilde{b} = T_1 \tilde{a}, \quad \tilde{c} = T_2 \tilde{b} \quad \Rightarrow \quad \tilde{c} = T_2 T_1 \tilde{a}.$$





3.3 旋转表示

■ 欧拉角 (RPY角)

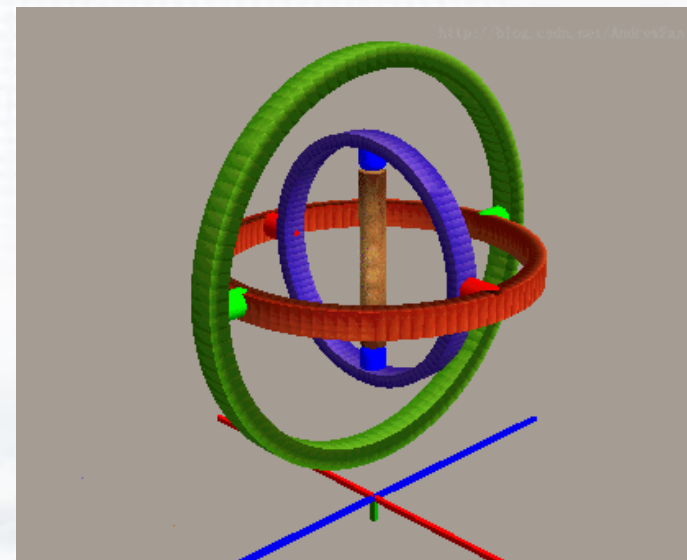
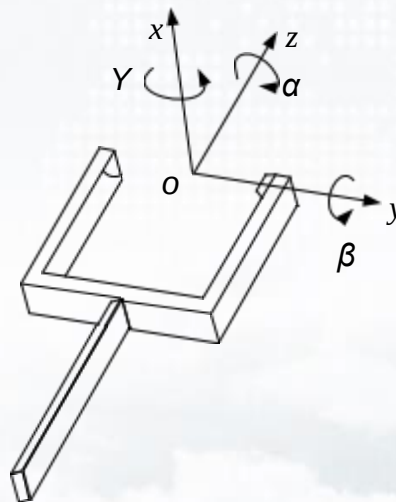
■ 坐标轴旋转可以用三个步骤来表示

绕 z 轴旋转 α 角: 滚动 (Roll)

绕 y 轴旋转 β 角: 俯仰 (Pitch)

绕 x 轴旋转 γ 角: 偏转 (Yaw)

■ 欧拉角表示的旋转矩阵



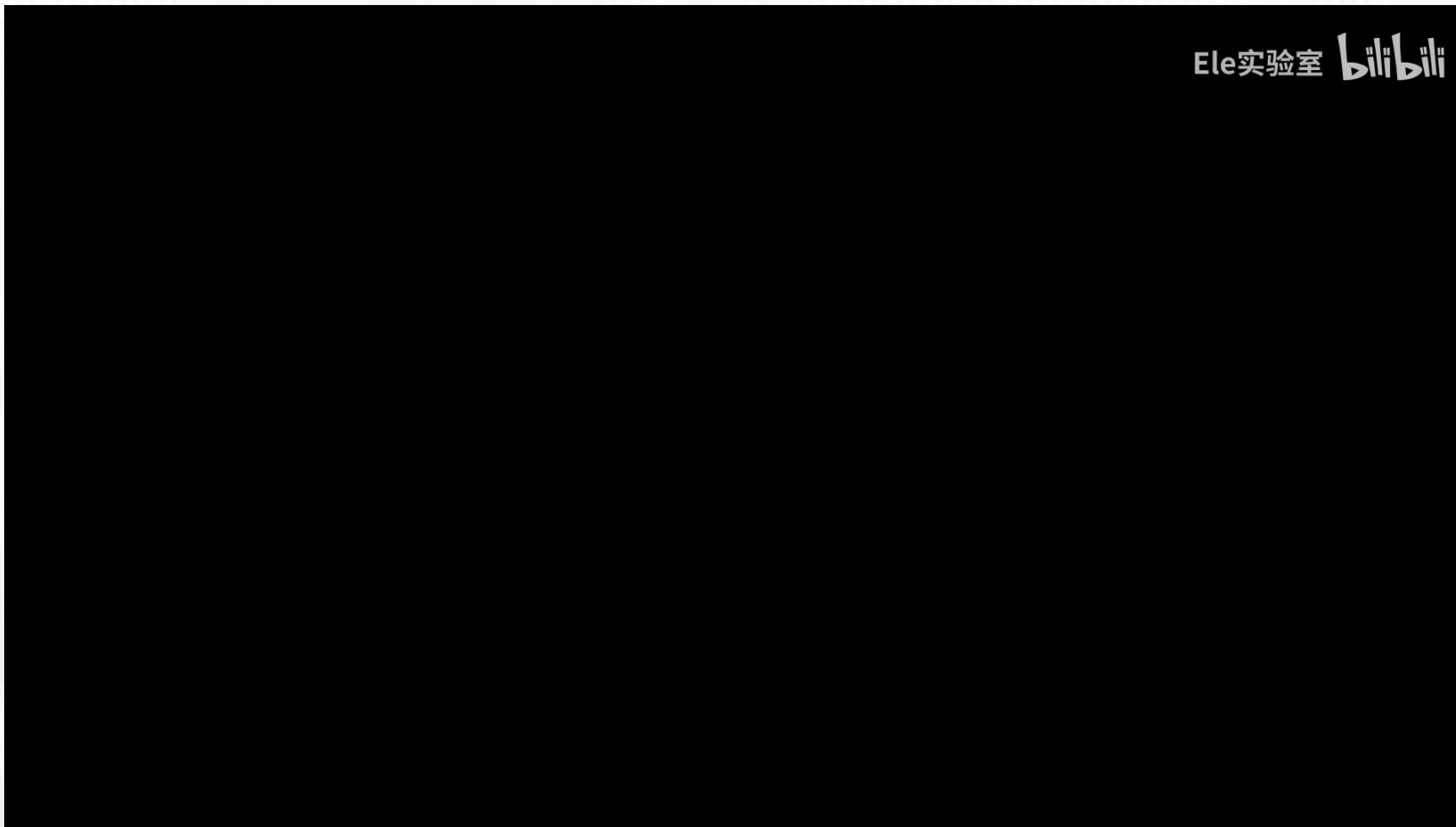
绕Z轴旋转

$$\begin{aligned} {}^A_B \mathbf{R}_{xyz}(\gamma, \beta, \alpha) &= \mathbf{R}(z_A, \alpha) \mathbf{R}(y_A, \beta) \mathbf{R}(x_A, \gamma) \\ &= \begin{bmatrix} c\alpha & -s\alpha & 0 \\ s\alpha & c\alpha & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} c\beta & 0 & s\beta \\ 0 & 1 & 0 \\ -s\beta & 0 & c\beta \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & c\gamma & -s\gamma \\ 0 & s\gamma & c\gamma \end{bmatrix} \end{aligned}$$



3.3 旋转表示

■ 欧拉角 (RPY角) : 存在万象节死锁问题



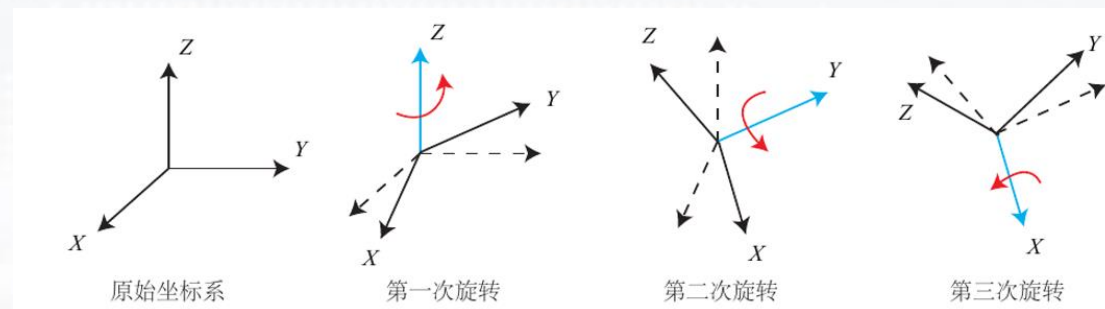


3.3 旋转表示

■ 欧拉角 (RPY角)

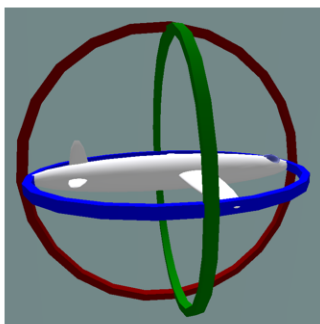
■ 万象锁问题

- 角位移分解为绕三个互相垂直轴的三次旋转组成的序列，工程中顺序难以确定

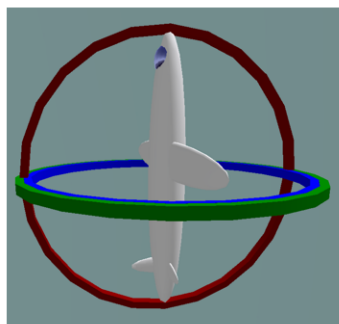


■ 在特定情况下出现奇异性，旋转自由度减1

正常情况



奇异情况



**由于万向锁的存在，欧拉角不适合插值或迭代
可以证明:仅用三个实数表达旋转时，不可避免地存在奇异性问题**



3.3 旋转表示

- **四元数 (Quaternion) : 用一个实部和三个虚部构成的复数向量表达旋转**
- **原理: 运用四维超球平面实现角度变换, 四元数变换和逆变换: 实部分两次完成旋转, 虚部抵消**

$$q = q_0 + q_1i + q_2j + q_3k \quad \text{满足: } i^2 = j^2 = k^2 = ijk = 1$$

- **优势: 形式紧凑、无奇异性**
 - 可实现连续插值, 解决万向节死锁 (Gimbal Lock) 问题
 - 仅需存储4个浮点数, 相比矩阵更加轻量
 - 四元数无论是求逆、串联等操作, 相比矩阵更加高效

视频理解:

https://www.bilibili.com/video/BV14t421h7M4/?share_source=copy_web&vd_source=efd7fbfe228a479045a3ba500e5d855f



3.4 TF2库

在机器人开发中，如何表示坐标系相对变换关系、计算模块间相对位姿？

■ TF2库

■ 核心功能

计算：相对位姿、坐标系变换

维护：维护坐标系间层级关系、实时更新变换关系

■ 应用场景

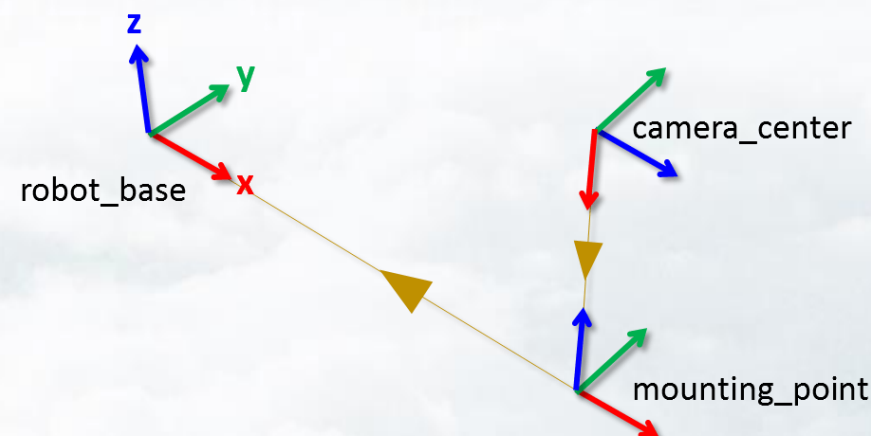
机器人导航：将激光雷达、相机数据转换到机器人参考系

传感器融合：统一多源数据，进行信息整合

运动控制：实时更新各部件相对位置，确保精度

工业自动化：支持多机器人及机械臂间的坐标协同

...





3.7 静态坐标变换

■ 静态坐标变换

运行期间不变的坐标关系

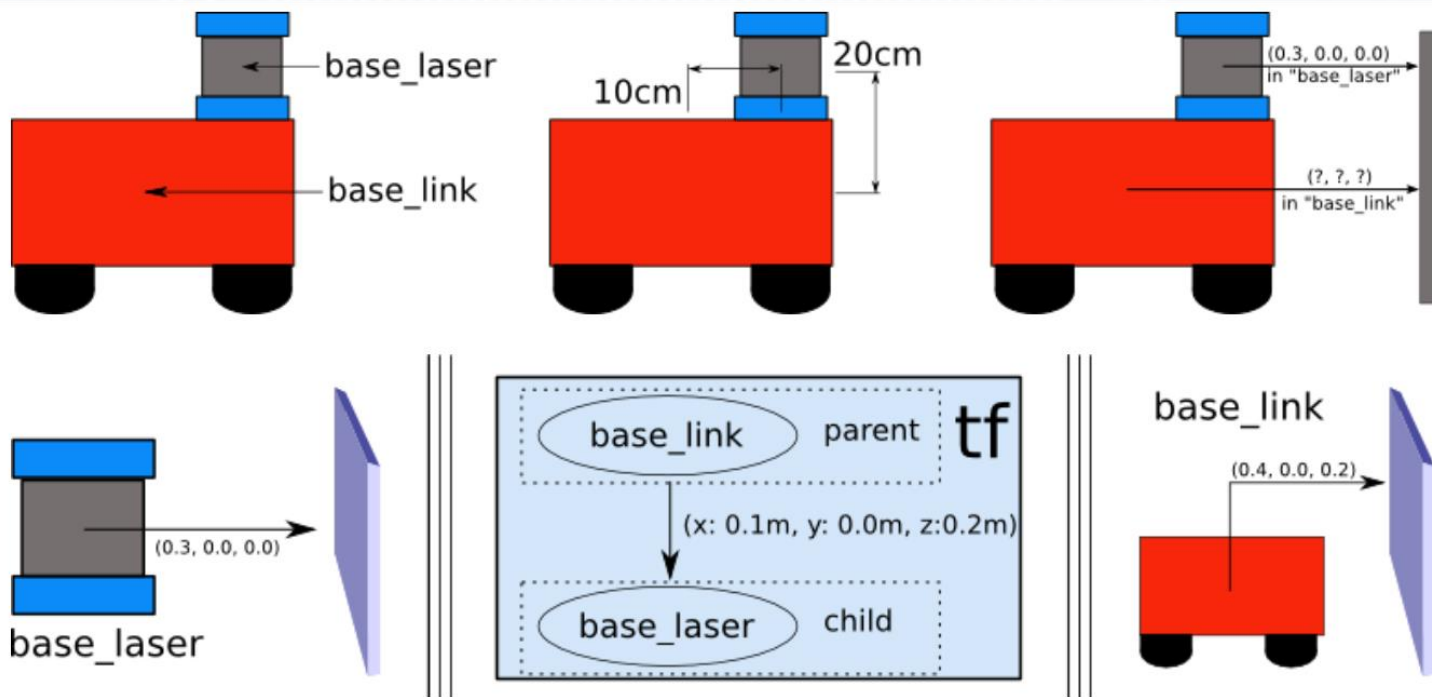
■ 特点

不随时间变化：变换数据一经设定，保持不变

资源消耗低：初始化加载后无需频繁计算

■ 典型应用

安装固定传感器（如激光雷达、摄像头）与机器人本体之间的坐标关系、机械臂各关节之间的固定相对位置



■ 实现方式

静态变换发布者（Static Transform Publisher）来发布这种固定变换。发布器在系统启动时将静态变换广播出去，其他节点订阅变换关系的消息



3.7 静态坐标变换

■ 案例：TF2静态变换广播

假设激光雷达固定安装在机器人前端，与机器人底盘（base_link）的相对位置固定。通过发布静态变换，整个系统在运行期间都能准确地知道激光雷达与底盘之间的关系，支持后续的数据融合、地图构建和定位。

```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws/src
ros2 pkg create static_tf2_example --build-type ament_python --dependencies rclpy tf2_ros geometry_msgs
```

修改 static_tf2_example 目录下的 setup.py 文件，添加入口点配置：

```
entry_points={
    'console_scripts': [
        'static_tf2_broadcaster = static_tf2_example.static_tf2_broadcaster:main'
    ],
}
```



3.7 静态坐标变换

static_tf2_broadcaster.py

案例：TF2静态变换广播

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import TransformStamped
from tf2_ros.static_transform_broadcaster import
StaticTransformBroadcaster
import math

class StaticFramePublisher(Node):
    def __init__(self):
        super().__init__('static_tf2_broadcaster')
        self.broadcaster = StaticTransformBroadcaster(self)
        self.publish_static_transform()
```

@staticmethod

def euler_to_quaternion(roll, pitch, yaw):

"""

将欧拉角 (roll, pitch, yaw) 转换为四元数 (x, y, z, w)。

"""

cy = math.cos(yaw * 0.5)

sy = math.sin(yaw * 0.5)

cp = math.cos(pitch * 0.5)

sp = math.sin(pitch * 0.5)

cr = math.cos(roll * 0.5)

sr = math.sin(roll * 0.5)

w = cr * cp * cy + sr * sp * sy

x = sr * cp * cy - cr * sp * sy

y = cr * sp * cy + sr * cp * sy

z = cr * cp * sy - sr * sp * cy

return (x, y, z, w)



3.7 静态坐标变换

static_tf2_broadcaster.py

def publish_static_transform(self):

定义一个 TransformStamped 消息

static_transform = TransformStamped()

static_transform.header.stamp = self.get_clock().now().to_msg()

static_transform.header.frame_id = "base_link" # 机器人底盘坐标系

static_transform.child_frame_id = "laser" # 激光雷达坐标系

设置平移部分 (单位: 米)

static_transform.transform.translation.x = 0.5

static_transform.transform.translation.y = 0.0

static_transform.transform.translation.z = 0.2

将欧拉角转换为四元数 (这里仅绕 Z 轴旋转, roll=0, pitch=0, yaw=0)

qtn = StaticFramePublisher.euler_to_quaternion(0, 0, 0)

将返回的四元数依次赋值给变换消息

static_transform.transform.rotation.x = qtn[0]

static_transform.transform.rotation.y = qtn[1]

static_transform.transform.rotation.z = qtn[2]

static_transform.transform.rotation.w = qtn[3]

发布静态变换

self.broadcaster.sendTransform(static_transform)

self.get_logger().info('Published static transform from
base_link to laser.')

def main(args=None):

rclpy.init(args=args)

node = StaticFramePublisher()

rclpy.spin(node)

node.destroy_node()

rclpy.shutdown()

if __name__ == '__main__':

main()



3.7 静态坐标变换

案例：TF2静态变换广播

```
cd ~/ros2_ws  
colcon build##编译
```

```
source install/setup.bash  
ros2 run static_tf2_example static_tf2_broadcaster
```

```
w@w-desktop: ~/ros2_ws  
[WARNING]: Unknown license 'TODO: License declaration'. This has been set in the  
package.xml, but no LICENSE file has been created.  
It is recommended to use one of the ament license identifiers:  
Apache-2.0  
BSL-1.0  
BSD-2.0  
BSD-2-Clause  
BSD-3-Clause  
GPL-3.0-only  
LGPL-3.0-only  
MIT  
MIT-0  
w@w-desktop:~/ros2_ws/src$ cd ..  
w@w-desktop:~/ros2_ws$ colcon build  
Starting >>> static_tf2_example  
Finished <<< static_tf2_example [2.59s]  
  
Summary: 1 package finished [2.97s]  
w@w-desktop:~/ros2_ws$ source install/setup.bash  
w@w-desktop:~/ros2_ws$ ros2 run static_tf2_example static_tf2_broadcaster  
[INFO] [1742021183.681526668] [static_tf2_broadcaster]: Published static transfo  
rm from base_link to laser.
```



3.7 静态坐标变换

案例：TF2静态变换广播

```
ros2 run tf2_ros tf2_echo base_link laser
```

#实时显示坐标变换信息

```
w@w-desktop: ~  
w@w-desktop:~$ ros2 run tf2_ros tf2_echo base_link laser  
[INFO] [1742021210.681223228] [tf2_echo]: Waiting for transform base_link -> la  
ser: Invalid frame ID "base_link" passed to canTransform argument target_frame -  
frame does not exist  
At time 0.0  
- Translation: [0.500, 0.000, 0.200]  
- Rotation: in Quaternion [0.000, 0.000, 0.000, 1.000]  
- Rotation: in RPY (radian) [0.000, -0.000, 0.000]  
- Rotation: in RPY (degree) [0.000, -0.000, 0.000]  
- Matrix:  
  1.000  0.000  0.000  0.500  
  0.000  1.000  0.000  0.000  
  0.000  0.000  1.000  0.200  
  0.000  0.000  0.000  1.000  
At time 0.0  
- Translation: [0.500, 0.000, 0.200]  
- Rotation: in Quaternion [0.000, 0.000, 0.000, 1.000]  
- Rotation: in RPY (radian) [0.000, -0.000, 0.000]  
- Rotation: in RPY (degree) [0.000, -0.000, 0.000]  
- Matrix:  
  1.000  0.000  0.000  0.500  
  0.000  1.000  0.000  0.000  
  0.000  0.000  1.000  0.200  
  0.000  0.000  0.000  1.000
```



3.8 动态坐标变换

■ 动态坐标变换

随时间变化的坐标系关系

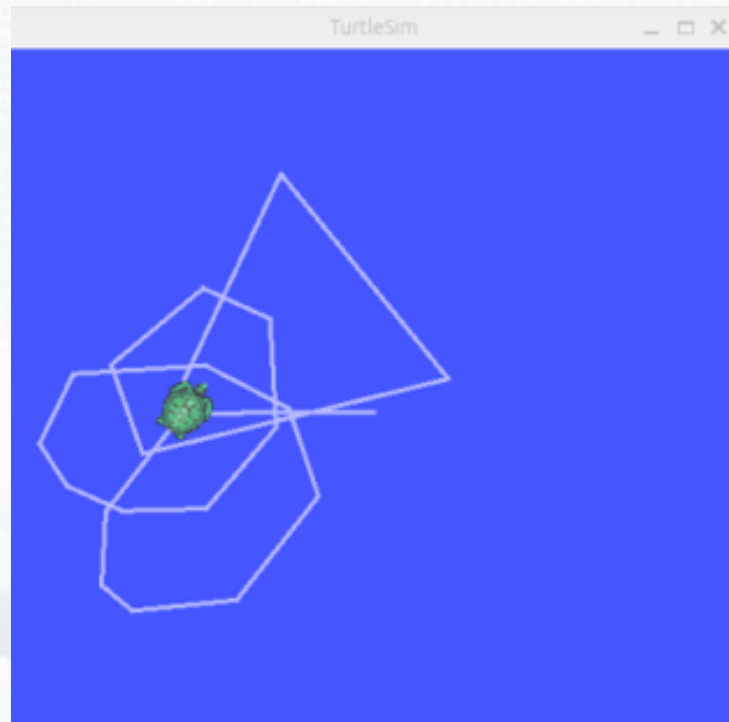
■ 特点

实时性要求高：变换数据必须保证低延迟和高准确性。

复杂性较高：需要对时间进行同步，处理时序数据。

■ 典型应用

机器人行走或移动时底盘与传感器之间的变换、机械臂运动过程中的各关节变换、动态环境下的物体跟踪等。



■ 实现方式

由 TF2 的 TransformBroadcaster 发布，其他节点通过 TF2 Buffer 与 TransformListener 订阅和查询变换数据。

系统会记录变换的历史数据，即使出现时延问题，也可以通过插值计算获得某一时刻准确的变换信息。



3.8 动态坐标变换

案例：基于Turtle-TF2动态变换广播

修改 static_tf2_example 目录下的 setup.py 文件，
添加入口点配置

```
entry_points={
    'console_scripts': [
        'static_tf2_broadcaster =
static_tf2_example.static_tf2_broadcaster:main',
        'turtle_tf2_broadcaster =
static_tf2_example.turtle_tf2_broadcaster:main',
        'turtle_tf2_listener =
static_tf2_example.turtle_tf2_listener:main'
    ],
},
```

```
ros2_ws/
├── src/
│   ├── static_tf2_example/
│   │   ├── package.xml
│   │   ├── setup.py
│   │   └── resource/
│   │       └── static_tf2_example
│   └── static_tf2_example/
│       ├── __init__.py
│       ├── static_tf2_broadcaster.py    # 静态广播节点: base_link -> laser
│       ├── turtle_tf2_broadcaster.py    # 动态广播节点: world -> turtle1
│       └── turtle_tf2_listener.py        # 动态监听节点: 查询 world -> turtle1 的变换
```



3.8 动态坐标变换

案例：动态变换广播节点 `turtle_tf2_broadcaster.py`

```
#!/usr/bin/env python3
import math
import rclpy
from rclpy.node import Node
from turtlesim.msg import Pose
from geometry_msgs.msg import TransformStamped
from tf2_ros.transform_broadcaster import TransformBroadcaster
class TurtleTF2Broadcaster(Node):
    def __init__(self):
        super().__init__('turtle_tf2_broadcaster')
        self.br = TransformBroadcaster(self)
        # 订阅 turtlesim 发布的位姿消息
        self.subscription = self.create_subscription(
            Pose,
            'turtle1/pose',
            self.pose_callback,
            10
        )
```

```
self.get_logger().info('Turtle TF2 broadcaster has been
started.')
```

```
@staticmethod
```

```
def euler_to_quaternion(roll, pitch, yaw):
```

```
    """
```

将欧拉角 (roll, pitch, yaw) 转换为四元数 (x, y, z, w)。
参数单位均为弧度。

```
    """
```

```
    cy = math.cos(yaw * 0.5)
```

```
    sy = math.sin(yaw * 0.5)
```

```
    cp = math.cos(pitch * 0.5)
```

```
    sp = math.sin(pitch * 0.5)
```

```
    cr = math.cos(roll * 0.5)
```

```
    sr = math.sin(roll * 0.5)
```

```
    w = cr * cp * cy + sr * sp * sy
```

```
    x = sr * cp * cy - cr * sp * sy
```

```
    y = cr * sp * cy + sr * cp * sy
```

```
    z = cr * cp * sy - sr * sp * cy
```

```
    return (x, y, z, w)
```



3.8 动态坐标变换

案例：动态变换广播节点 `turtle_tf2_broadcaster.py`

`def pose_callback(self, msg):`

```
# 构造 TransformStamped 消息
t = TransformStamped()
t.header.stamp = self.get_clock().now().to_msg()
t.header.frame_id = "world"    # 全局坐标系
t.child_frame_id = "turtle1"   # 小乌龟坐标系
# 设置平移部分 (turtlesim 的 x, y 坐标)
t.transform.translation.x = msg.x
t.transform.translation.y = msg.y
t.transform.translation.z = 0.0
# 利用小乌龟的朝向 msg.theta 转换为四元数 (仅绕 Z 轴旋转)
q = TurtleTF2Broadcaster.euler_to_quaternion(0, 0,
msg.theta)
```

```
t.transform.rotation.x = q[0]
t.transform.rotation.y = q[1]
t.transform.rotation.z = q[2]
t.transform.rotation.w = q[3]
# 广播变换
self.br.sendTransform(t)
self.get_logger().info(f'Broadcast transform: x={msg.x:.2f},
y={msg.y:.2f}, theta={msg.theta:.2f}')

def main(args=None):
    rclpy.init(args=args)
    node = TurtleTF2Broadcaster()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```




3.8 动态坐标变换

案例：动态变换监听节点

turtle_tf2_listener.py

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from tf2_ros.buffer import Buffer
from tf2_ros.transform_listener import TransformListener
from tf2_ros import LookupException, ConnectivityException,
ExtrapolationException
```

```
class TurtleTF2Listener(Node):
```

```
    def __init__(self):
```

```
        super().__init__('turtle_tf2_listener')
```

```
        self.tf_buffer = Buffer()
```

```
        self.tf_listener = TransformListener(self.tf_buffer, self)
```

```
        # 定时器每秒查询一次变换
```

```
        self.timer = self.create_timer(1.0, self.timer_callback)
```

```
        self.get_logger().info('Turtle TF2 listener has been
started.')
```

```
    def timer_callback(self):
```

```
        try:
```

```
            now = rclpy.time.Time()
```

```
            trans = self.tf_buffer.lookup_transform('world', 'turtle1', now)
```

```
            self.get_logger().info(
```

```
                f"Transform: translation ({trans.transform.translation.x:.2f}, "
```

```
                f"{trans.transform.translation.y:.2f},
```

```
                {trans.transform.translation.z:.2f}), "
```

```
                f"rotation ({trans.transform.rotation.x:.2f},
```

```
                {trans.transform.rotation.y:.2f}, "
```

```
                f"{trans.transform.rotation.z:.2f},
```

```
                {trans.transform.rotation.w:.2f})"
```

```
            )
```

```
        except (LookupException, ConnectivityException,
ExtrapolationException) as e:
```

```
            self.get_logger().info('Could not transform: %s' % str(e))
```

```
def main(args=None):
```

```
    rclpy.init(args=args)
```

```
    node = TurtleTF2Listener()
```

```
    rclpy.spin(node)
```

```
    node.destroy_node()
```

```
    rclpy.shutdown()
```

```
if __name__ == '__main__':
```

```
    main()
```



3.8 动态坐标变换 案例：动态变换广播

```
colcon build
source install/setup.bash
```

启动 turtlesim 节点（用于动态演示）：

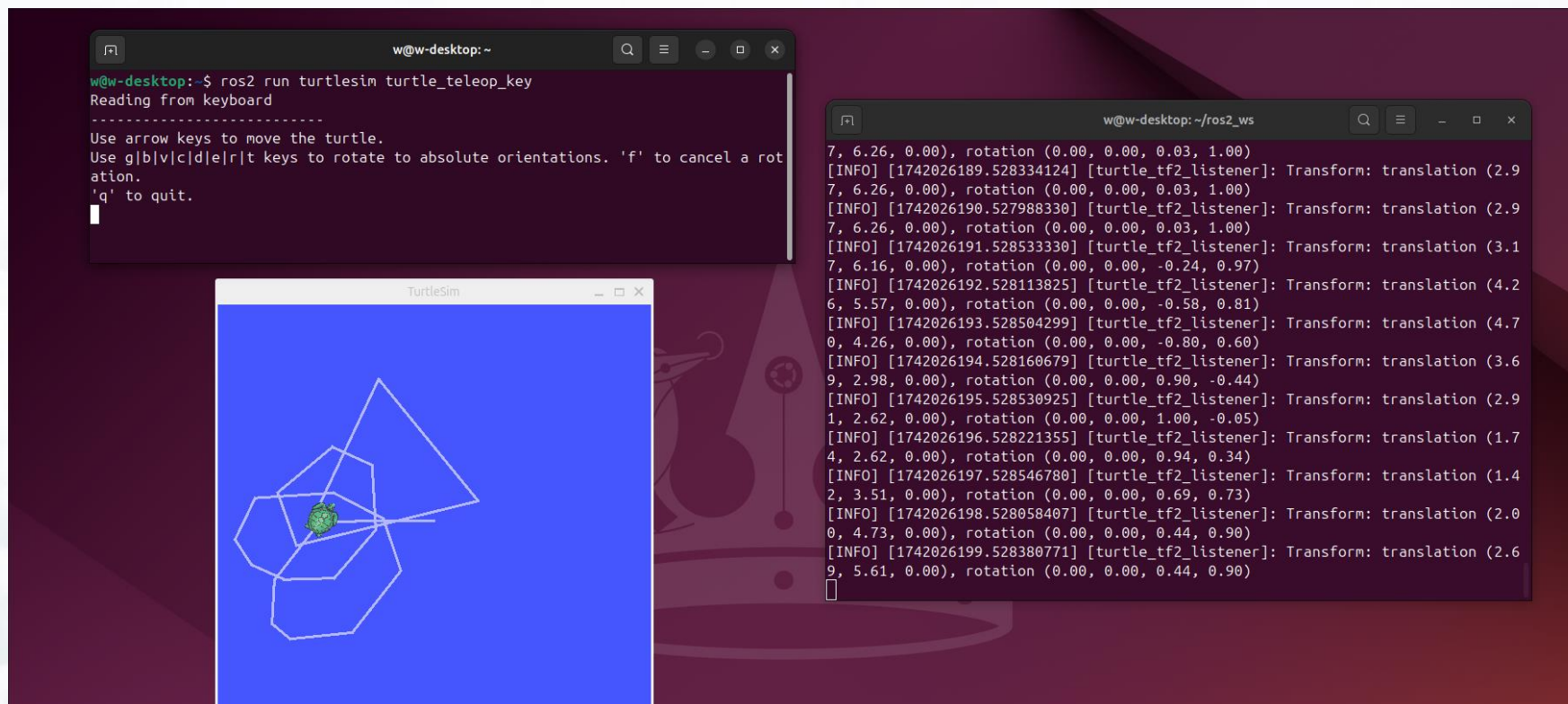
```
ros2 run turtlesim turtlesim_node
ros2 run turtlesim turtle_teleop_key
```

启动动态广播节点（world -> turtle1）：

```
ros2 run static_tf2_example turtle_tf2_broadcaster
```

启动动态监听节点（监听并打印 world -> turtle1 变换）：

```
ros2 run static_tf2_example turtle_tf2_listener
```





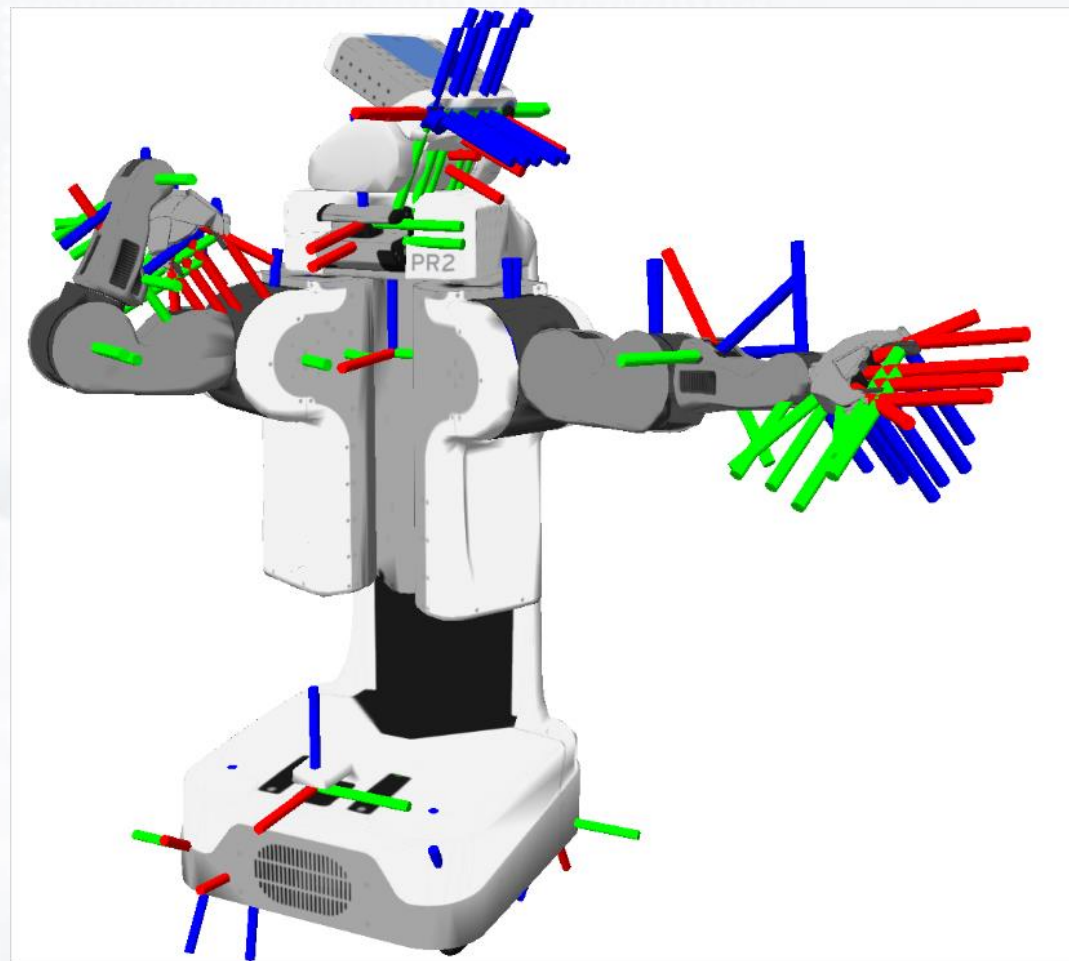
3.9 多坐标系变换

■ 多坐标系变换

上述例子的坐标变换仅在两个坐标系之间
还需要多个坐标系之间的变换方法

世界坐标系

- └─ 机器人基坐标系
- └─ 相机坐标系
- └─ 雷达坐标系
- └─ 机械臂坐标系





3.9 多坐标系变换

案例：多坐标系变换

配置setup.py

```
ros2_ws/  
└─ src/  
    └─ static_tf2_example/  
        ├── package.xml  
        ├── setup.py  
        └── resource/  
            └─ static_tf2_example  
└─ launch/  
    ├── multi_tf2_launch.py # 启动静态发布器的 launch 文件  
    └─ static_tf2_example/  
        ├── __init__.py  
        └─ multi_tf2_listener.py # 多坐标系监听节点
```

```
from glob import glob  
data_files=[  
    # 安装资源文件和 package.xml  
    ('share/ament_index/resource_index/packages',  
    ['resource/' + package_name]),  
    ('share/' + package_name, ['package.xml']),  
    # launch 文件  
    ('share/' + package_name + '/launch',  
    glob('launch/*.py')),  
],  
  
entry_points={  
    'console_scripts': [  
        # 多坐标系监听节点入口  
        'multi_tf2_listener =  
static_tf2_example.multi_tf2_listener:main'  
    ],  
}
```



3.9 多坐标系变换

多坐标系变换发布

launch/multi_tf2_launch.py

```
#!/usr/bin/env python3
import os
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        # 发布从 world 到 base_link 的变换
        Node(
            package='tf2_ros',
            executable='static_transform_publisher',
            name='world_to_base_link',
            arguments=['1.0', '0.0', '0.0', '0', '0', '0', '1', 'world',
'base_link']
        ),
    ])
```

TO

发布从 base_link 到 laser 的变换

```
Node(
    package='tf2_ros',
    executable='static_transform_publisher',
    name='base_link_to_laser',
    arguments=['0.2', '0.0', '0.1', '0', '0', '0', '1', 'base_link', 'laser']
),
```

发布从 base_link 到 camera 的变换

```
Node(
    package='tf2_ros',
    executable='static_transform_publisher',
    name='base_link_to_camera',
    arguments=['0.0', '0.2', '0.15', '0', '0', '0', '1', 'base_link', 'camera']
),
```

发布从 base_link 到 arm 的变换

```
Node(
    package='tf2_ros',
    executable='static_transform_publisher',
    name='base_link_to_arm',
    arguments=['-0.2', '0.0', '0.0', '0', '0', '0', '1', 'base_link', 'arm']
)
])
```




多坐标系监听节点

multi_tf2_listener.py

3.9 多坐标系变换

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from tf2_ros.buffer import Buffer
from tf2_ros.transform_listener import TransformListener
from tf2_ros import LookupException,
ConnectivityException, ExtrapolationException

class MultiTF2Listener(Node):
    def __init__(self):
        super().__init__('multi_tf2_listener')
        self.tf_buffer = Buffer()
        self.tf_listener = TransformListener(self.tf_buffer, self)
        # 每2秒查询一次各个变换
        self.timer = self.create_timer(2.0, self.timer_callback)
        self.get_logger().info("Multi TF2 listener started.")
    def timer_callback(self):
        pairs = [
            ("world", "base_link"),
            ("base_link", "laser"),
            ("base_link", "camera"),
            ("base_link", "arm")
        ]
```

```
for parent, child in pairs:
    try:
        trans = self.tf_buffer.lookup_transform(parent, child,
rclpy.time.Time())
        self.get_logger().info(
            f"Transform from {parent} to {child}: "
            f"translation = ({trans.transform.translation.x:.2f}, "
            f"{trans.transform.translation.y:.2f}, "
            f"{trans.transform.translation.z:.2f}), "
            f"rotation = ({trans.transform.rotation.x:.2f}, "
            f"{trans.transform.rotation.y:.2f}, "
            f"{trans.transform.rotation.z:.2f}, "
            f"{trans.transform.rotation.w:.2f})"
        )
    except (LookupException, ConnectivityException,
ExtrapolationException) as e:
        self.get_logger().warn(f"Could not lookup transform from
{parent} to {child}: {str(e)}")
def main(args=None):
    rclpy.init(args=args)
    node = MultiTF2Listener()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
if __name__ == '__main__':
    main()
```




3.9 多坐标系变换

案例：多坐标系变换

```
colcon build  
source install/setup.bash
```

启动 launch 文件发布静态变换：

```
ros2 launch static_tf2_example multi_tf2_launch.py
```

启动多坐标系监听节点

```
ros2 run static_tf2_example multi_tf2_listener
```

```
w@w-desktop: ~/ros2_ws
[static_transform_publisher-2] translation: ('0.200000', '0.000000', '0.100000')
[static_transform_publisher-2] rotation: ('0.000000', '0.000000', '0.000000', '1.000000')
[static_transform_publisher-2] from 'base_link' to 'laser'
[static_transform_publisher-3] [INFO] [1742028702.038686883] [base_link_to_camera]: Spinning until stopped - publishing transform
[static_transform_publisher-3] translation: ('0.000000', '0.200000', '0.150000')
[static_transform_publisher-3] rotation: ('0.000000', '0.000000', '0.000000', '1.000000')
[static_transform_publisher-3] from 'base_link' to 'camera'
[static_transform_publisher-1] [INFO] [1742028702.045142593] [world_to_base_link]: Spinning until stopped - publishing transform
[static_transform_publisher-1] translation: ('1.000000', '0.000000', '0.000000')
[static_transform_publisher-1] rotation: ('0.000000', '0.000000', '0.000000', '1.000000')
[static_transform_publisher-1] from 'world' to 'base link'
[static_transform_publisher-4] [INFO] [1742028702.049349961] [base_link_to_arm]: Spinning until stopped - publishing transform
[static_transform_publisher-4] translation: ('-0.200000', '0.000000', '0.000000')
[static_transform_publisher-4] rotation: ('0.000000', '0.000000', '0.000000', '1.000000')
[static_transform_publisher-4] from 'base_link' to 'arm'

w@w-desktop: ~/ros2_ws
_link: translation = (1.00, 0.00, 0.00), rotation = (0.00, 0.00, 0.00, 1.00)
[INFO] [1742028960.314051172] [multi_tf2_listener]: Transform from base_link to laser: translation = (0.20, 0.00, 0.10), rotation = (0.00, 0.00, 0.00, 1.00)
[INFO] [1742028960.315591012] [multi_tf2_listener]: Transform from base_link to camera: translation = (0.00, 0.20, 0.15), rotation = (0.00, 0.00, 0.00, 1.00)
[INFO] [1742028960.316788017] [multi_tf2_listener]: Transform from base_link to arm: translation = (-0.20, 0.00, 0.00), rotation = (0.00, 0.00, 0.00, 1.00)
[INFO] [1742028962.312791079] [multi_tf2_listener]: Transform from world to base_link: translation = (1.00, 0.00, 0.00), rotation = (0.00, 0.00, 0.00, 1.00)
[INFO] [1742028962.314075103] [multi_tf2_listener]: Transform from base_link to laser: translation = (0.20, 0.00, 0.10), rotation = (0.00, 0.00, 0.00, 1.00)
[INFO] [1742028962.315789814] [multi_tf2_listener]: Transform from base_link to camera: translation = (0.00, 0.20, 0.15), rotation = (0.00, 0.00, 0.00, 1.00)
[INFO] [1742028962.316894078] [multi_tf2_listener]: Transform from base_link to arm: translation = (-0.20, 0.00, 0.00), rotation = (0.00, 0.00, 0.00, 1.00)
[INFO] [1742028964.312874774] [multi_tf2_listener]: Transform from world to base_link: translation = (1.00, 0.00, 0.00), rotation = (0.00, 0.00, 0.00, 1.00)
[INFO] [1742028964.313879111] [multi_tf2_listener]: Transform from base_link to laser: translation = (0.20, 0.00, 0.10), rotation = (0.00, 0.00, 0.00, 1.00)
[INFO] [1742028964.315166006] [multi_tf2_listener]: Transform from base_link to camera: translation = (0.00, 0.20, 0.15), rotation = (0.00, 0.00, 0.00, 1.00)
[INFO] [1742028964.316242566] [multi_tf2_listener]: Transform from base_link to arm: translation = (-0.20, 0.00, 0.00), rotation = (0.00, 0.00, 0.00, 1.00)
```

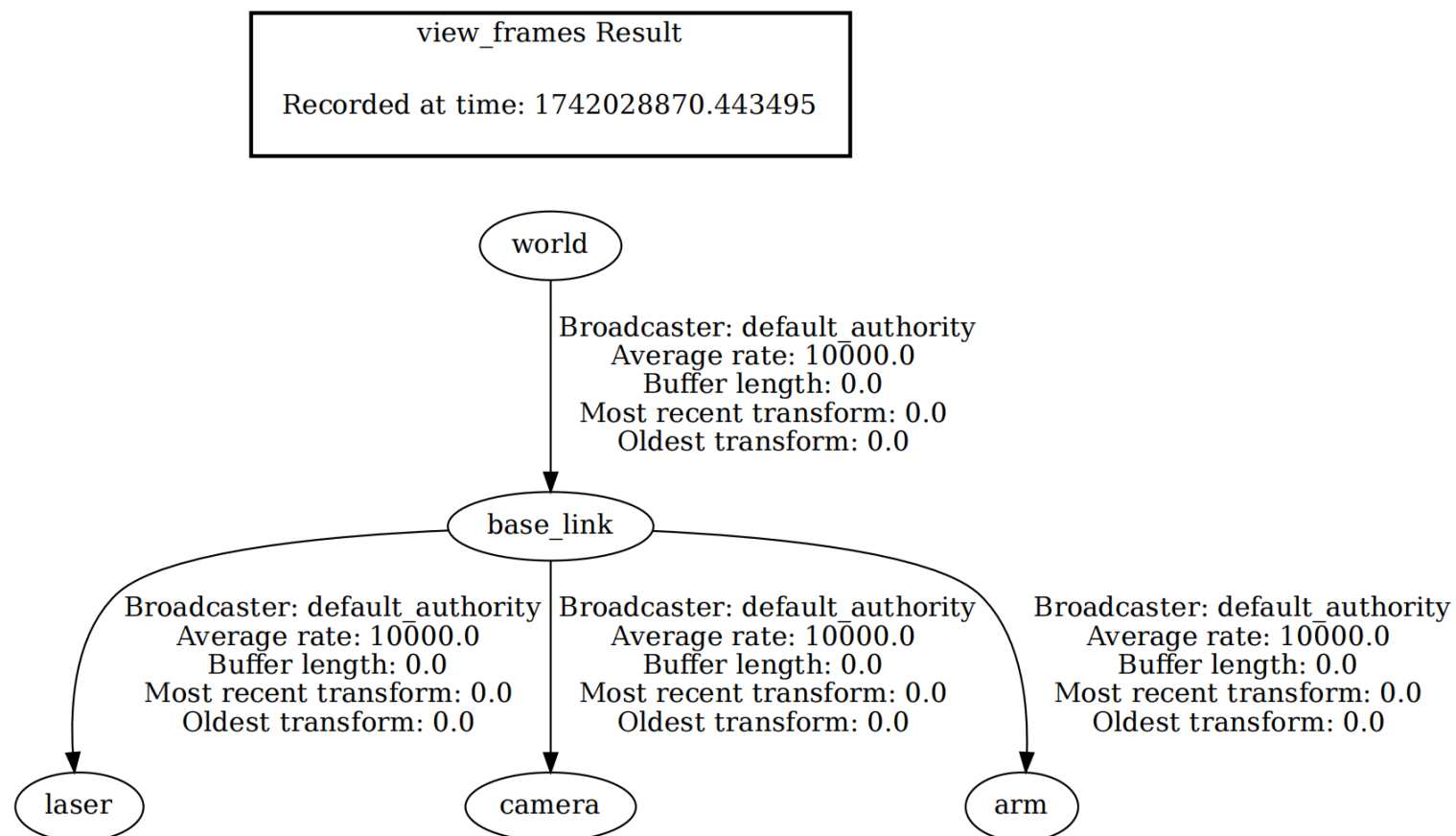


3.9 多坐标系变换 案例：多坐标系变换

TF Tree 图

使用 tf2 提供的工具生成 TF 树图：

```
ros2 run tf2_tools view_frames
```





机器人仿真方法



同济大学
TONGJI UNIVERSITY



土木工程学院
COLLEGE OF CIVIL ENGINEERING



1.1 仿真定义

■ 为什么需要仿真？

- 硬件依赖：需要真实的机器人、传感器和计算设备，**成本高**
- 环境限制：现实世界中难以搭建**复杂测试场景**（城市、工厂）
- 安全风险：测试可能导致**硬件损坏或人员受伤**
- 调试困难：**现实环境变量多**，重复测试耗时且不一致

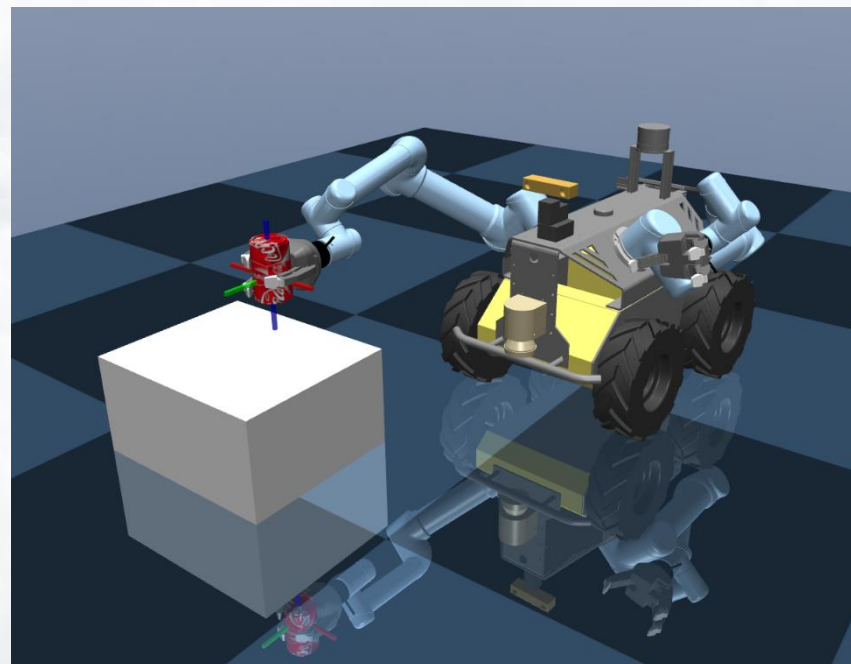
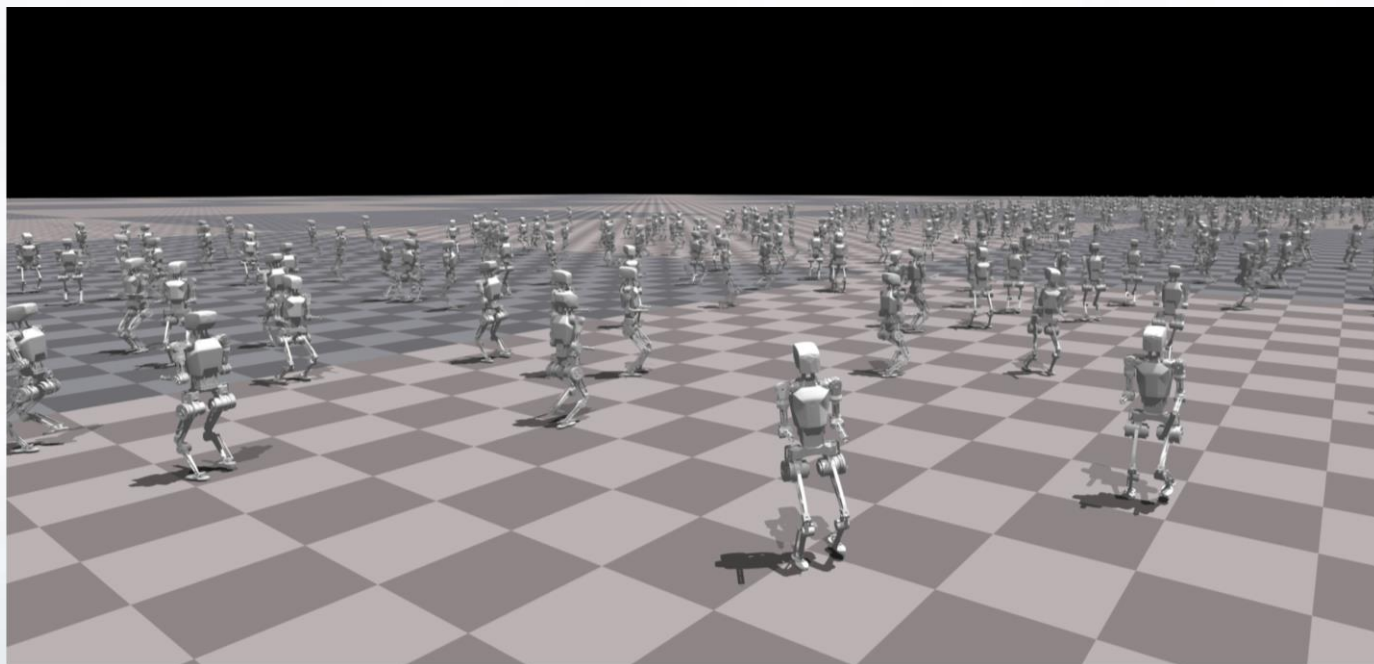


先丢下水看看
啥情况



1.1 仿真定义

- **仿真：**泛指基于实验或训练的目的，将原本的系统、事务或流程，建立一个模型以表征其关键特性(Key Characteristics)或者行为/功能，予以系统化与公式化，以便对关键特征做出模拟。
- **机器人仿真：**通过计算机模拟机器人的运动、感知和决策过程以评估其性能和行为。





1.2 仿真过程

■ 仿真对象

■ 机器人

- 机器人模型：URDF、STL 定义结构和外观

■ 传感器

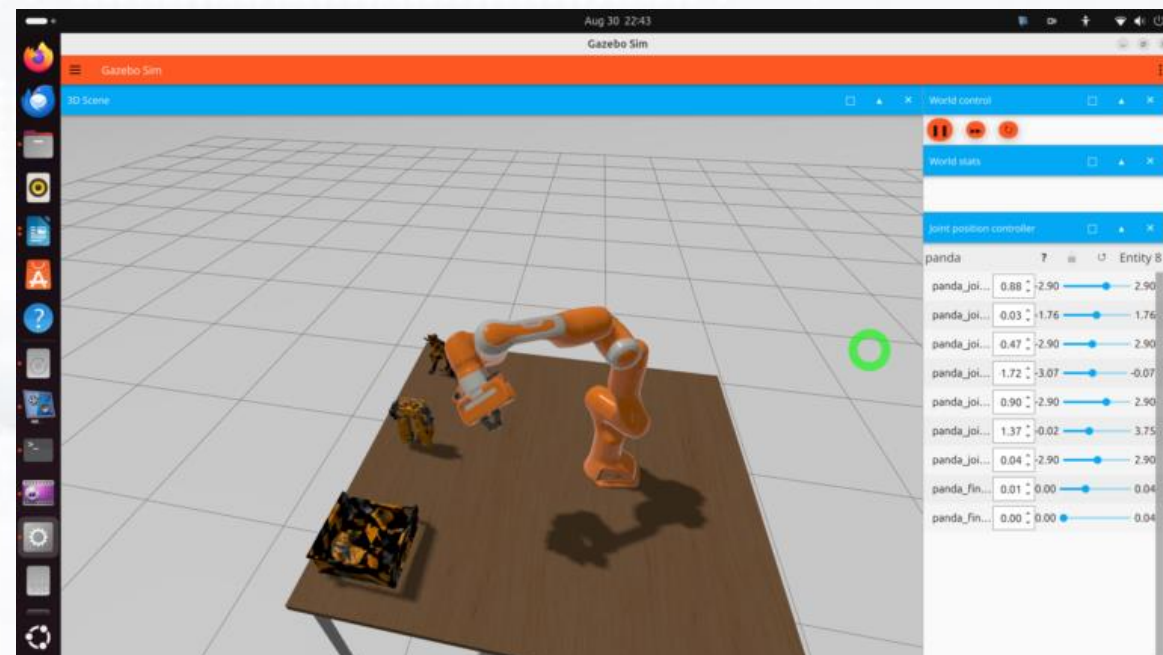
- 传感器：激光雷达、摄像头、IMU

■ 场景环境

- 环境：静态/动态物体、光源、地形

■ 仿真目的

- 机器人行为交互与控制的关键特性和模拟
- 机器人系统设计和功能验证
- 机器人各项性能评估与分析





1.2 仿真过程

由此可以将仿真过程拆分为以下几个步骤：

■ step1-数学建模：

- 将机器人的每个组件（如连杆、关节和效应器）都通过**数学模型**来表示。
- 几何形状、动力学（力和运动的关系）、静力学（静止或慢速运动中的力的平衡） ...

■ step2-物理仿真：

- 仿真过程的核心
- 用数学模型来模拟真实世界的**物理法则**，如重力、摩擦、碰撞和材料属性。

■ step3-感知与环境模拟：

- 使用**模拟传感器**（如摄像头、激光雷达、接触传感器）收集环境信息
- 在**虚拟环境**中创建与真实世界相似的条件（如光照变化、障碍物、不同的表面纹理）
- 在仿真环境中模拟出传感器的输入-输出

■ step4-控制系统集成：

- 机器人**行为控制**：处理传感器数据和决定机器人的动作
- 基于不同的算法、策略，令控制系统直接作用于机器人的数学模型上，控制其虚拟表示的行动

■ step5-观察与迭代优化：

- 直观的图形化界面，开发者能够观察机器人行为，进行迭代优化调整



1.3 仿真原理

■ 如何在计算机中模拟现实世界中的连续运动？

- 质点/刚体：质量 m 、初始力 F 、位置 x 、速度 v
- 运动模式：
 - 平动(Translation)：质心运动代表
 - 转动(Rotation)：同时绕轴旋转，考虑旋转属性（角速度 ω 、角位移 θ ）
- 基于牛顿第二定律计算加速度： $a = F/m$
- 更新速度： $v(t + \delta t) = v(t) + a * \delta t$
- 更新位置： $x(t + \delta t) = x(t) + v(t + \delta t) * \delta t$

■ 数值积分方法：将连续的时间域离散化，在每步迭代中更新物体状态

- 欧拉(Euler)法
- 龙格-库塔(Runge-Kutta)法:其中四阶(RK4)是最常用的方法之一，计算四个不同点的斜率的加权平均来预测下一时间步的值
- Verlet积分：常见于分子动力学
- 多步法：如亚当斯-巴什福斯方法和亚当斯-莫尔顿方法
-

■ 积分方法的适用性

- 欧拉方法最简单，但精度受步长影响显著，且在处理刚体旋转问题时，直接对欧拉角积分会引入“万向节死锁”问题
- 旋转矩阵与四元数：能够有效避免奇点问题

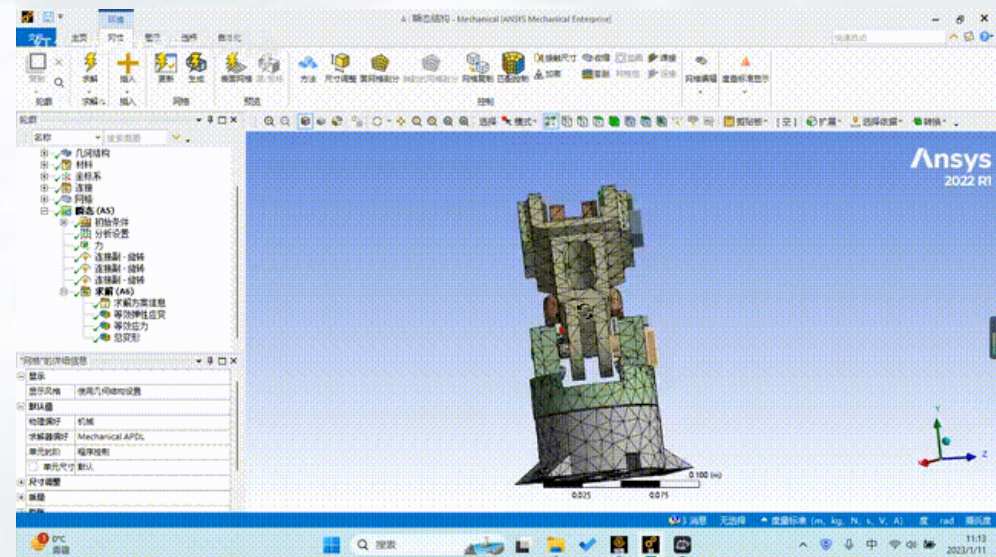
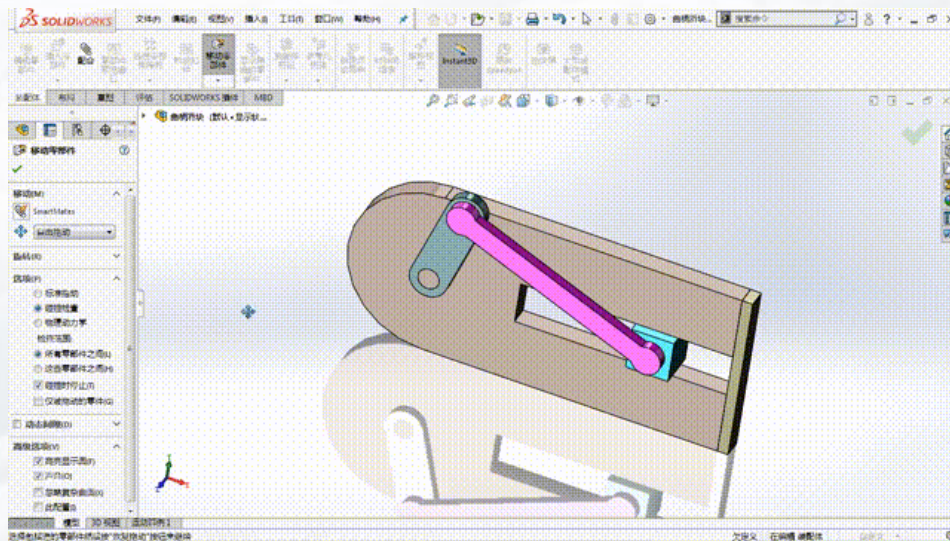
太复杂了！交给物理引擎吧！

1.4 仿真的核心组件

物理引擎：专门设计来模拟物理现象的软件库：重力、碰撞检测/响应、刚体/软体动力学、流体动态.....

核心特点：

- **碰撞检测与响应：**物理引擎能够检测仿真环境中的物体何时相互接触，并计算出相应的物理反应，如反弹、摩擦等。
- **动力学仿真：**通过数值积分方法来模拟物体在力的作用下的运动，包括线性运动和角动量守恒等。
- **约束和关节：**支持多种类型的约束和关节，允许创建复杂的机械结构和模拟其运动。
- **数值稳定性：**使用高度稳定的积分方法来保证模拟过程的稳定性和精确性。
- **性能优化：**提供不同的求解器，以根据需求在精度和速度间进行取舍。





1.4 仿真的核心组件

Gazebo: 一个开源的 3D 机器人仿真器，用于模拟机器人、传感器和环境

起源: 2002 年由 Andrew Howard 和 Nate Koenig 在南加州大学 (USC) 创建，最初为学术研究工具，用于机器人仿真。早期与 Player/Stage 项目（机器人中间件）集成，独立于 ROS

现在: Gazebo 和 ROS/ROS 2 由 Open Robotics 统一维护

Gazebo工具简介

- 独立于ROS/ROS2的工具，有独立的物理等引擎，无需 ROS 2 或 ROS 框架
- 目前实现了ROS2与Gazebo的高度集成
- ROS 2 提供通信和控制框架（节点、话题、服务），Gazebo 提供物理和传感器仿真，二者通过接口协作。

Gazebo已经成为ROS/ROS2仿真的首选工具



1.4 仿真的核心组件

Gazebo: 机器人仿真的核心工具

一个开源的 3D 机器人仿真器，用于模拟机器人、传感器和环境

支持:

物理仿真 (碰撞、动力学)

传感器仿真 (激光雷达、摄像头)

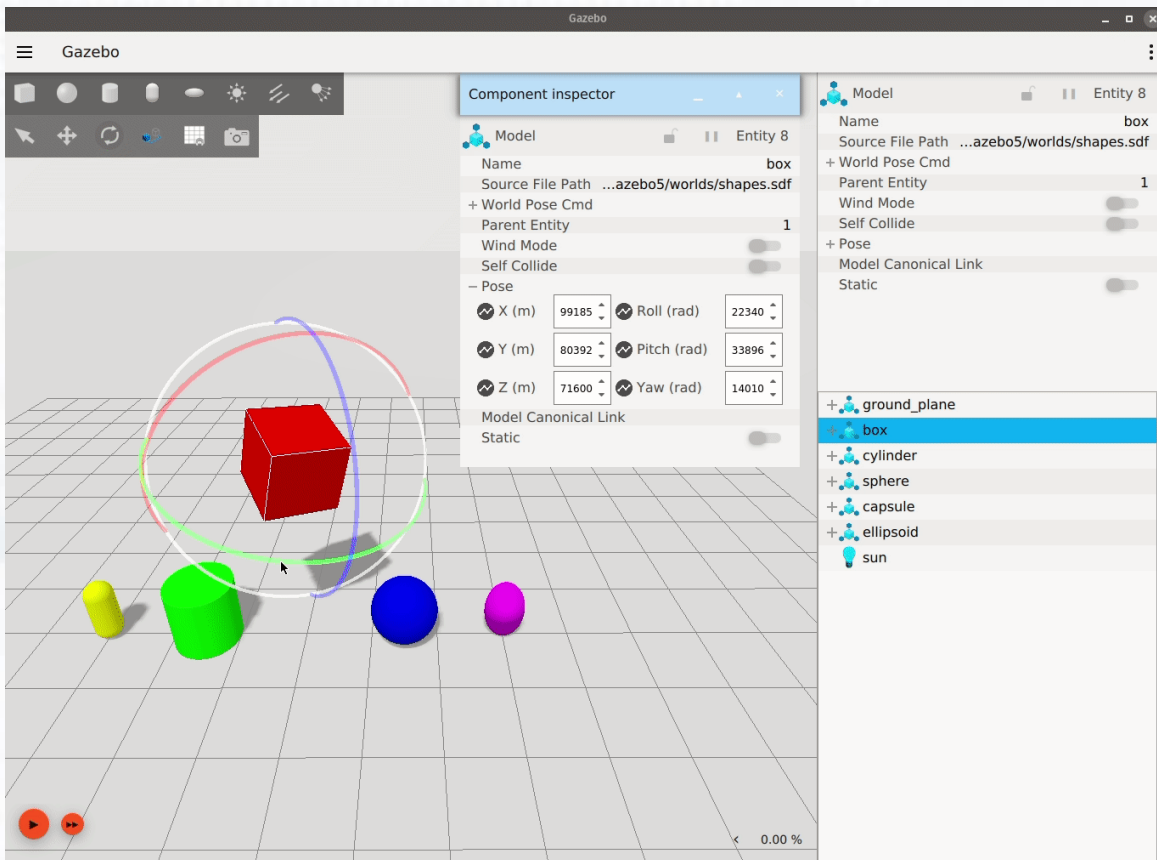
场景建模

核心功能:

物理引擎: 模拟重力、摩擦、碰撞 (ODE、Bullet、DART)

渲染引擎: 生成视觉效果和传感器数据 (OGRE、OGRE2)

ROS 集成: 通过话题和服务与 ROS 2 通信



Gazebo Harmonic
图形化界面



1.4 仿真的核心组件

Gazebo: 机器人仿真的核心工具

Gazebo

左侧工具栏

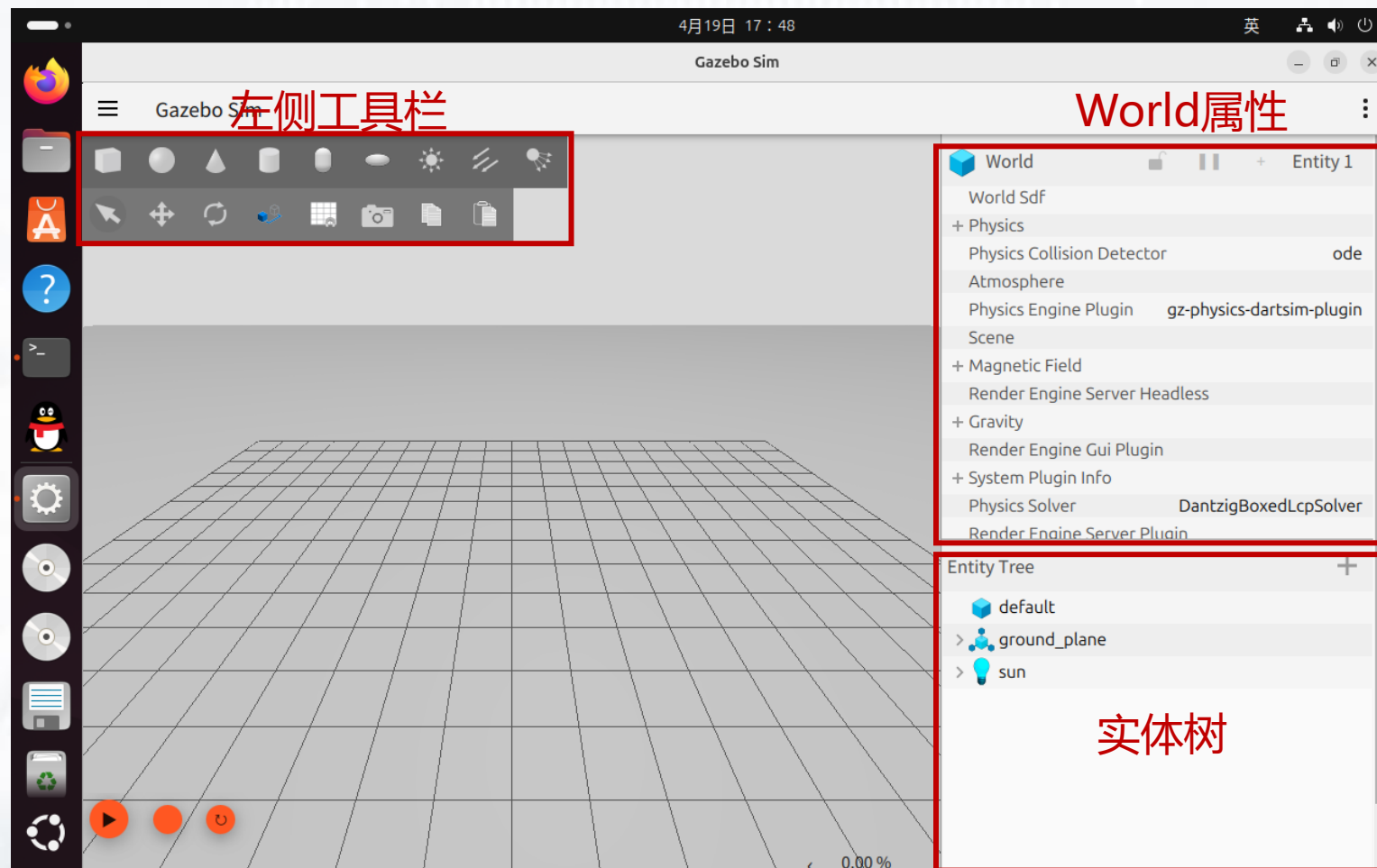
添加基础图形
移动缩放等工具
添加文件、光照等

World属性栏

仿真世界的全局设置
物理引擎 (Physics)
渲染设置 (Render Engine)
光照 (Lighting) 等
可以调整重力、光照等参数

实体树

场景中的所有对象 (实体)
如地面 (ground_plane)
光源 (sun) 等





1.4 仿真的核心组件

Rviz2: ROS2的官方可视化工具

用于显示机器人模型、传感器数据、坐标变换等

Rviz2工具简介:

ROS Visualization

一款显示多种数据的三维可视化平台，帮助我们更直观地感受系统中的数据及其计算过程

-实现所有可监测信息的图形化显示

Rviz2如何实现数据可视化

数据: 各种调试机器人时常用的数据，比如：图像数据、三维点云数据、地图数据、机器人模型数据等

可视化: 直观的看到数据，比如一个三维的点(100,100,100),通过RVIZ可以将其显示在空间中

不同数据的可视化: 强大的插件，可以显示各种**数据**类型



1.4 仿真的核心组件：Rviz2常用显示功能总览

名称	功能	消息类型
Axes	显示 RViz2 默认的坐标系（三轴箭头）	—
Camera	显示带内参的相机图像	sensor_msgs/msg/Image, sensor_msgs/msg/CameraInfo
Grid	显示网格线（参考系原点为中心，辅助定位）	—
Grid Cells	显示网格中被占用的单元格（如局部成本地图）	nav_msgs/msg/GridCells
Image	显示图像（不依赖 CameraInfo）	sensor_msgs/msg/Image
InteractiveMarker	显示交互式标记，可进行鼠标拖动交互	visualization_msgs/msg/InteractiveMarker
Laser Scan	显示激光雷达扫描数据（2D 扫描）	sensor_msgs/msg/LaserScan
Map	显示静态地图或 SLAM 地图数据	nav_msgs/msg/OccupancyGrid
Markers	显示任意几何图形，如箭头、线、文本等	visualization_msgs/msg/Marker, MarkerArray
Path	显示路径信息（如导航路径）	nav_msgs/msg/Path
PointStamped	显示一个点（小球）	geometry_msgs/msg/PointStamped
Pose	显示单个位姿（箭头或坐标轴表示）	geometry_msgs/msg/PoseStamped
PoseArray	显示多个位姿组成的数组	geometry_msgs/msg/PoseArray
PointCloud2	显示点云数据	sensor_msgs/msg/PointCloud, PointCloud2
Polygon	显示多边形轮廓线（如障碍区域）	geometry_msgs/msg/Polygon
Odometry	显示里程计数据（轨迹、姿态等）	nav_msgs/msg/Odometry
Range	显示超声波或红外测距传感器的范围锥体	sensor_msgs/msg/Range
RobotModel	显示 URDF 定义的机器人模型	—
TF	显示 TF 坐标变换树（父子坐标关系）	—
Wrench	显示力和力矩（箭头+圆圈）	geometry_msgs/msg/WrenchStamped
Oculus	将 RViz 场景渲染到 Oculus 设备中	—



1.4 仿真的核心组件 Rviz2: ROS2的官方可视化工具

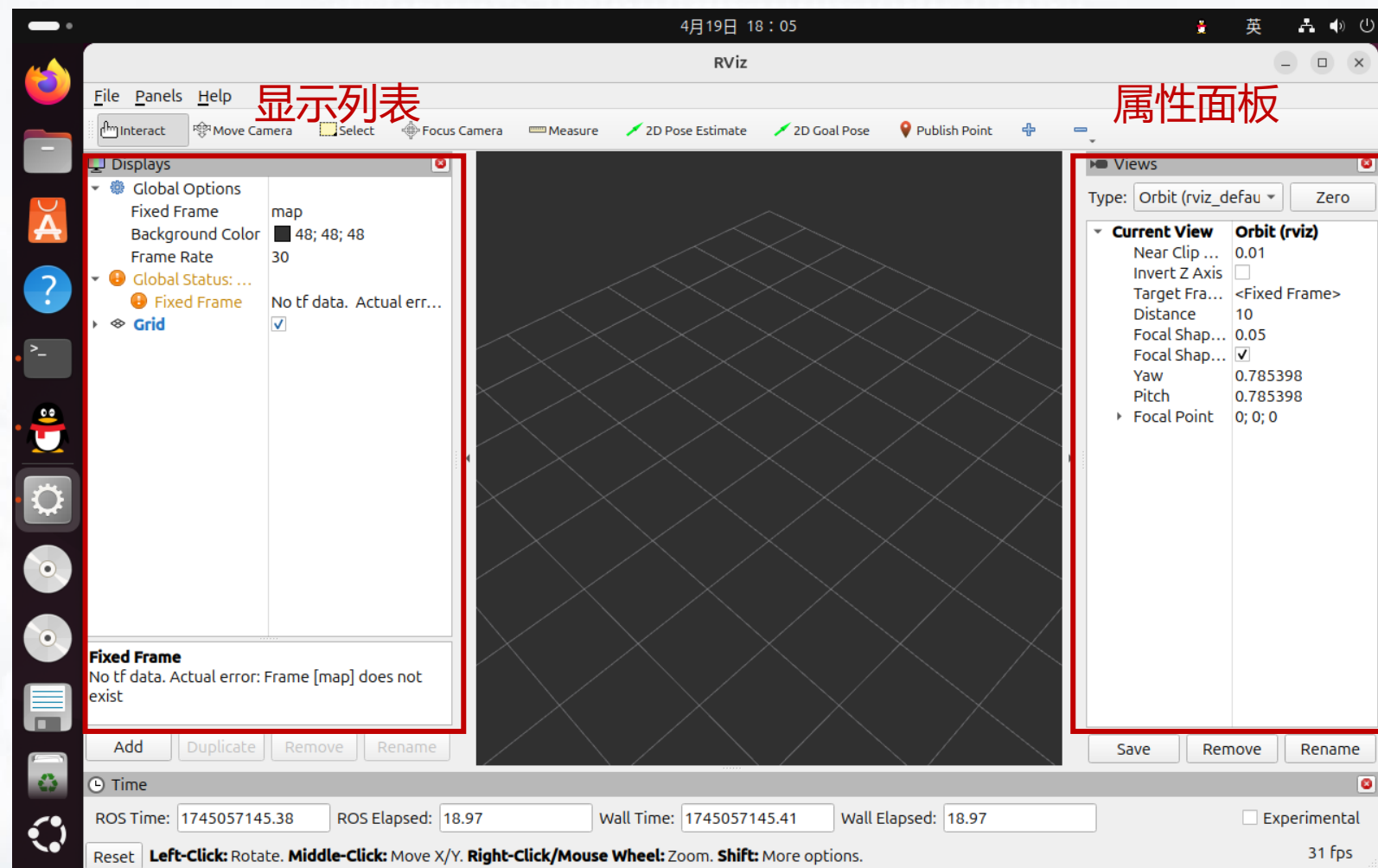
Rviz2 GUI

显示列表

添加和管理可视化元素

属性面板

调整显示设置





机器人建模语言



同济大学
TONGJI UNIVERSITY



土木工程学院
COLLEGE OF CIVIL ENGINEERING



2.1 机器人建模

思考：我们需要描述机器人系统的哪些东西？

- 从硬件分类上看，有：**控制器、传感器、驱动器、执行器**
- 从描述内容上看，主要是为数学建模服务的物理属性
 - 几何尺寸
 - 质量
 - 关节类型、约束
 -

- 机器人通常被建模为由连杆和关节组成的结构

连杆：带有质量属性的刚体

关节：连接、限制两个刚体相对运动的结构，也可称为运动副

通过关节将连杆依次连接起来，就构成了一个个运动链（也就是我们一般意义上的机器人模型）





2.2 统一机器人描述格式

URDF: Unified Robot Description Format

- 提供一种尽可能通用的机器人描述规范
- 基于XML通用语法
- 基本构成如下:

```
<!--pr2 robots' urdf file-->
1 <?xml version="1.0"?>
2 <?xml-model href="https://raw.githubusercontent.com/ros/urdfdom/master/xsd/urdf.xsd" ?>
3 <robot name="pr2" xmlns="http://www.ros.org">
4   <link> ... </link>
5   <link> ... </link>
6   <link> ... </link>
7
8   <joint> .... </joint>
9   <joint> .... </joint>
10  <joint> .... </joint>
11 </robot>
```

核心概念：标签
URDF预定义了一系列标签



2.2 统一机器人描述格式

在机器人数学建模中，需要将机器人的物理结构、运动学和动力学抽象为数学量，以支持仿真（Gazebo）、可视化（RViz）和控制（ROS 2）

具体描述内容

几何形状：描述机器人连杆的形状和尺寸，用于外观（可视化）和碰撞检测（物理仿真）

位置与朝向：指定连杆、关节或传感器的空间位置（xyz 坐标）和旋转（rpy 滚转-俯仰-偏航，弧度制）

质量：指定连杆的质量（单位：千克），描述其重量

惯性：指定连杆的惯性矩阵（ $ixx, iyy, izz, ixy, iyz, ixz$ ），描述其对旋转的阻力

关节类型与运动轴：指定关节的类型（固定、旋转、连续旋转、滑动）和运动轴（xyz 向量）

运动范围：指定关节的运动限制，如旋转角度（弧度）或滑动距离（米）

材质：指定连杆的外观颜色或纹理（RGBA 或纹理文件）

.....

这些数学量通过 URDF 文件定义



2.2 统一机器人描述格式

概述:

URDF 文件使用 XML 标签定义机器人模型, 核心标签分为

连杆相关 (<link> 及其子标签)

关节相关 (<joint> 及其子标签)

材质 (<material>)

仿真配置 (<gazebo>)

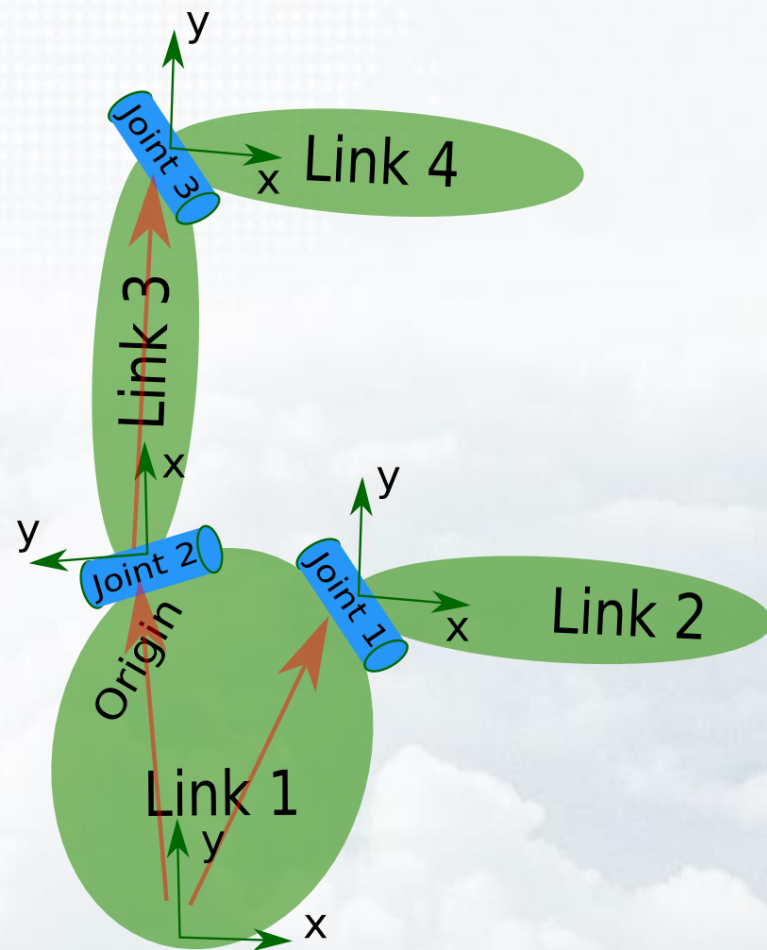
分类:

连杆相关标签: <link> 及其子标签 (<visual>、<collision>、<inertial>、<geometry>、<material>、<origin>)

关节相关标签: <joint>、<parent>、<child>、<axis>、<limit>、<origin>

仿真配置标签: <gazebo> (含 <plugin>)

根标签: <robot> (包含所有标签)



机器人结构示意图



2.3 URDF标签

标签	作用	子标签 / 属性
<robot>	根标签，定义机器人模型的名称与结构	name (机器人名称)
<link>	定义连杆 (刚体部件，如底盘、轮子)	<visual>, <collision>, <inertial>
<joint>	定义关节，连接两个连杆	name, type, <parent>, <child>, <origin>, <axis>
<visual>	连杆的可视化几何 (用于 RViz/Gazebo 渲染)	<geometry>, <material>, <origin>
<collision>	连杆的碰撞几何 (用于物理仿真)	<geometry>, <origin>
<inertial>	连杆的惯性属性 (质量、惯性矩阵)	<mass>, <inertia>, <origin>
<geometry>	几何类型定义 (用于 visual 或 collision)	<box>, <cylinder>, <sphere>, <mesh>
<material>	定义颜色或纹理 (用于视觉渲染)	name, <color>
<origin>	定义坐标变换: 位置与姿态	xyz, rpy (弧度制)
<axis>	定义关节的旋转或移动轴	xyz (轴向量)
<transmission>	连接关节与控制器 (仿真控制使用)	<actuator>, <joint>
<gazebo>	Gazebo 仿真配置 (如插件、传感器)	<plugin>, <sensor> 等



2.4 <robot> 标签

作用:

作为 URDF 文件的根标签，所有其他标签（如 <link>、<joint>）必须包含在 <robot> 内
定义整个机器人模型的名称和结构

属性:

name: 机器人模型的名称（字符串）

主 URDF 文件中 必须 指定 name 属性

在被包含的文件中，name 是可选的；若指定，需与主文件的 name 一致

子标签:

包含所有其他标签，如 <link>、<joint>

示例:

```
<!-- 根标签 -->
<robot name="my_robot">
  <!-- 其他标签 -->
  <link>
    <!-- 其他标签 -->
  </link>
</robot>
```



2.5 <link> 标签

作用：描述的是刚体(rigid body)的惯性、视觉特征和碰撞特性

属性：name：连杆的唯一名称（字符串，必填）

<link> 是父标签，直接嵌套以下子标签：

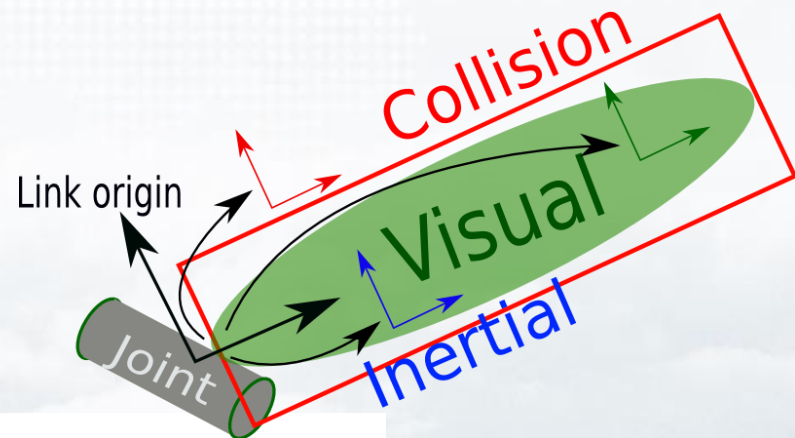
<visual>：外观（可选）

<collision>：碰撞几何（可选）

<inertial>：质量和惯性（可选，动力学仿真需要）

示例：

```
<link name="robot_arm">
  <inertial>
    <origin xyz="0 0.1 0" rpy="0 0 0"/>
    <mass value="1.5"/>
    <inertia ixx="0.01" ixy="0" ixz="0" iyy="0.02" iyz="0" izz="0.015"/>
  </inertial>
  <visual>
    ...
  </visual>
  <collision>
    ...
  </collision>
</link>
```





2.5.1 <inertial> 标签

作用：定义连杆的质量和惯性，用于 Gazebo 的动力学仿真（如计算运动和力）

子标签：

- <mass>（必填）：定义连杆质量（单位：千克）
- <origin>（可选）：定义质心相对于连杆坐标系的位置和方向
 - xyz属性定义质心的位置
 - rpy (roll, pitch, yaw) 属性定义旋转
 - 未指定的话即为原点(恒等变换)(defaults to identity if not specified)
- <inertia>（如果需要运动则必填）：定义惯性矩阵，描述连杆绕质心的 x、y、z 轴转动惯量
 - I_{xx}, I_{yy}, I_{zz} 是质量惯性矩(moments of inertia),计算公式： $I = \int r^2 dm$
 - I_{xy}, I_{yz}, I_{xz} 是惯性积(products of inertia),计算公式： $I_{xy} = \int xy dm$

意义：

一个简单的机器人手臂，可以想象由多个长方体或者圆柱体部件组成

每个部件的动态行为不仅取决于其自身的质量和尺寸，还取决于其相对于旋转轴的分布

当机器人手臂移动时，每个关节的扭矩都必须克服连接到该关节的所有下游部件的惯性

因此，计算每个部件的惯性矩阵对于设计机器人的控制系统非常重要

计算公式-官方速查文档: https://en.wikipedia.org/wiki/List_of_moments_of_inertia



2.5.1 <inertial> 标签

示例:

```
<link name="robot_arm">
  <inertial>
    <origin xyz="0 0.1 0" rpy="0 0 0"/>
    <mass value="1.5"/>
    <inertia ixx="0.01" ixy="0" ixz="0" iyy="0.02" iyz="0" izz="0.015"/>
  </inertial>
  <visual>
    ...
  </visual>
  <collision>
    ...
  </collision>
</link>
```




2.5.2 <visual> 标签

作用：定义连杆的外观（形状、颜色），用于 RViz 和 Gazebo 可视化

属性：name（可选）：视觉元素的名称，建议命名以区分多个 <visual>

子标签：

- <origin>（可选）：**定义相对于连杆坐标系的偏移和旋转**
 - xyz属性定义偏移量
 - rpy (roll, pitch, yaw) 属性定义偏转量
 - 未指定的话即为原点(恒等变换)(defaults to identity if not specified)
- <geometry>（必填）：定义形状，支持四种类型：
 - <box size="x y z"/>：盒状立方体，由长宽高三个属性定义
 - <cylinder radius="r" length="l"/>：圆柱体，由基面圆半径和高度定义
 - <sphere radius="r"/>：球体，由半径定义
 - <mesh filename="package://path/model.dae" scale="x y z"/>：mesh形状网格文件
 - 推荐使用.dae格式(collada,支持颜色和纹理，描述更精确)，此外.stl等也能支持
 - scale: 网格的缩放因子，用于调整网格模型的大小（缩放，例如等比缩放一半即为scale="0.5 0.5 0.5"）



2.5.2 <visual> 标签

子标签:

<material> (可选) : 定义颜色或纹理

<color rgba="r g b a"/> : RGBA 颜色 (0-1)

<texture filename="package://path/texture.png"/> : 纹理文件

可在 <robot> 顶级定义, 供多个 <link> 共享

```
<robot name="my_robot">
<!-- 在顶级<robot>元素中定义材质, 以便可以在多个<link>之间共享 -->
<material name="carbon_fiber">
  <!-- 定义颜色 -->
  <color rgba="1 1 1 1"/>
  <!-- 使用白色作为基色, 这通常是应用纹理时的做法 -->
  <!-- 定义纹理 -->
  <texture filename="package://my_robot_package/meshes/textures/carbon_fiber_texture.png"/>
</material>
<link name="bottom_board">
  <visual>
    <geometry>
      <box size="1 1 0.1"/>
    </geometry> <!-- 引用上面定义的材质 -->
    <material name="carbon_fiber"/>
  </visual>
</link>
</robot>
```



2.5.3 <collision> 标签

作用：定义连杆的碰撞几何，用于 Gazebo 物理仿真（如碰撞检测、接触力计算）

嵌套在 <link> 内，描述连杆在仿真中的碰撞体积

属性：name（可选）：碰撞元素的名称，建议命名以区分多个 <collision>

子标签：

- <origin>（可选）：定义碰撞几何相对于连杆坐标系的偏移和旋转
 - xyz属性定义偏移量
 - rpy（roll, pitch, yaw）属性定义偏转量
 - 未指定的话即为原点(恒等变换)(defaults to identity if not specified)
- <geometry>（必填）：定义碰撞形状，与 <visual> 用法相同：
 - <box size="x y z"/>：长方体。
 - <cylinder radius="r" length="l"/>：圆柱。
 - <sphere radius="r"/>：球体。
 - <mesh filename="package://path/model.dae" scale="x y z"/>：mesh网格文件（.dae 或 .stl）



值得说明的是：为了提高仿真性能，碰撞几何通常应该比视觉几何更简化

以SW可视化举例：

- 对于一个视觉上比较复杂的部件模型
- 其碰撞模型可能只是一个或几个基本几何形状的组合
- 用矩形/圆柱体进行外接包络



案例：<link>定义方法：以ugv rover为例

思考：如何定义这个机器人的link?

拆分

官方设计，中间2个轮悬空，为被动轮

四角的轮负责驱动





案例：<link>定义方法：以ugv rover为例

因此，机器人的主体部分明确

下面为base_link部分代码的定义

```
<link
  name="base_link">
  <inertial>                                描述base_link的惯性属性
    <origin
      xyz="-0.000460087791654118 -6.50856019576702E-10 -0.0147959620149623"
      rpy="0 0 0" />
    <mass                                指定惯性坐标系相对于base_link坐标系的位姿
      value="2.78375064359473" />
    <inertia                                指定base_link的质量
      ixx="0.00651567554169617"
      ixy="-5.86655425789789E-13"
      ixz="3.77886436056975E-05"
      iyy="0.00933991752413667"
      iyz="-2.53116742521487E-11"
      izz="0.0121583506563461" />
  </inertial>                                描述转动惯量
```




案例：<link>定义方法：以ugv rover为例

下面为base_link部分代码的定义

```
<visual>                                描述base_link的可视化几何形状
  <origin
    xyz="0 0 0"
    rpy="0 0 0" />                      指定可视化几何体的坐标系相对于base_link坐标系的位置
  <geometry>
    <mesh                                描述可视化几何体的形状
      filename="package://ugv_description/meshes/ugv_rover/base_link.stl" />
    </geometry>
    <material
      name=""
      <color                              指定可视化几何体的材质和颜色
        rgba="0.752941176470588 0.752941176470588 0.752941176470588 1" />
      </material>
    </visual>
  <collision>                            描述base_link的碰撞几何体
    <origin
      xyz="0 0 0"
      rpy="0 0 0" />                    指定碰撞几何体的坐标系相对于base_link坐标系的位置
    <geometry>
      <mesh                              描述碰撞几何体的形状
        filename="package://ugv_description/meshes/ugv_rover/base_link.stl" />
      </geometry>
    </collision>
  </link>
```



案例：<link>定义方法：以ugv rover为例

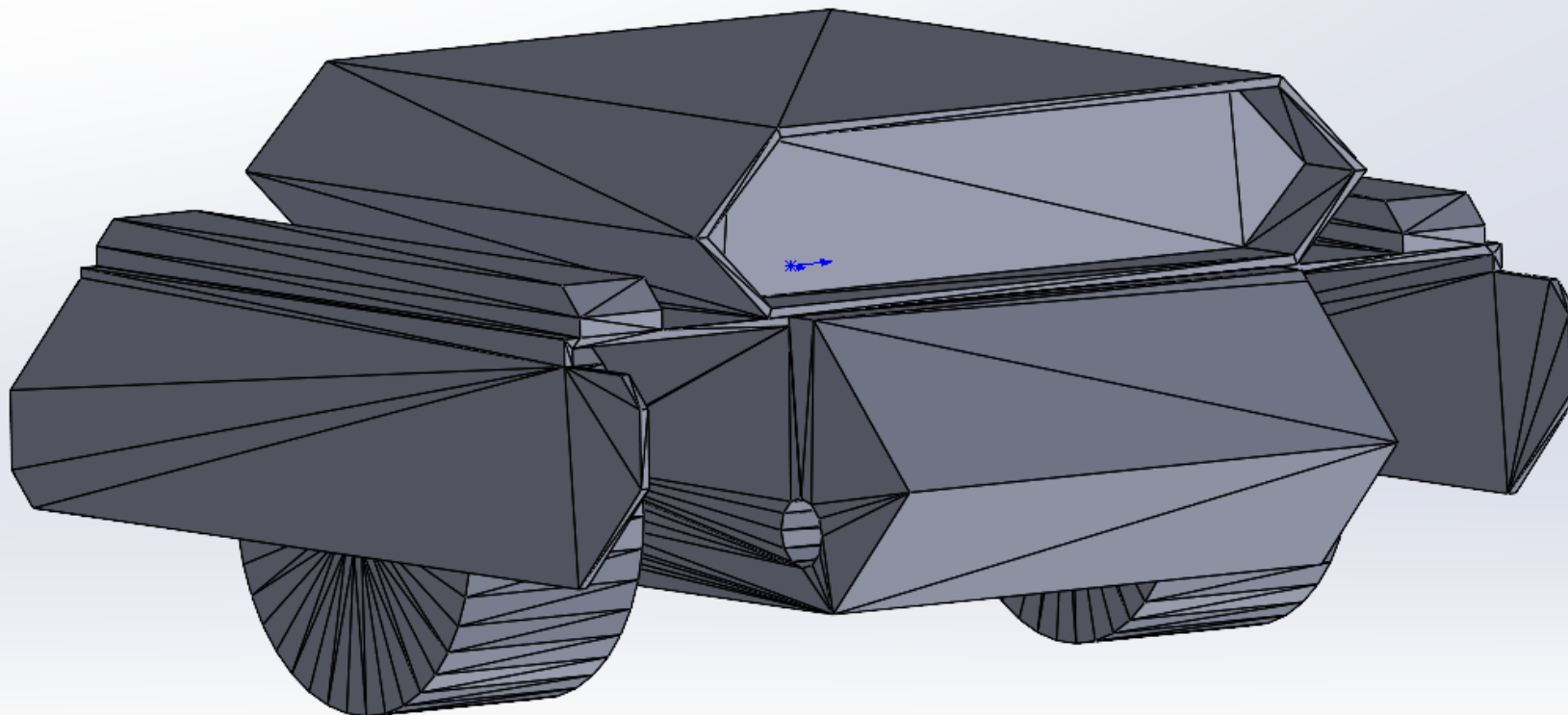
base_link.stl加载显示

[wvshareteam repositories · GitHub](#)

包括：

ugv模型文件

Roarm模型文件





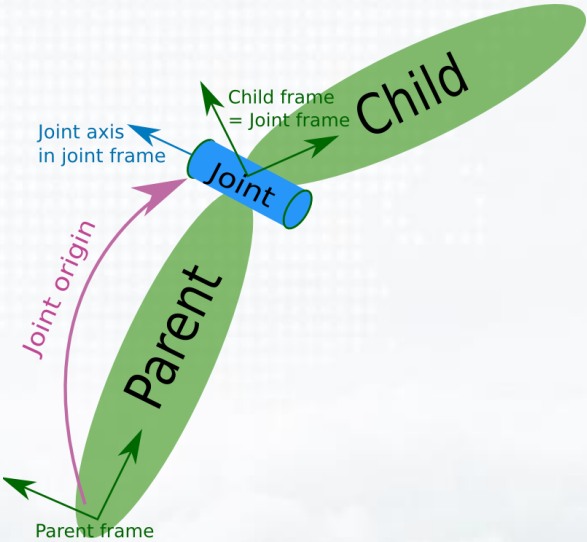
2.6 <joint> 标签

作用： <joint> 用于描述机器人中 两个 link 之间的运动连接关系，包括运动类型、运动范围、运动轴方向等信息

属性：

name: 关节的名称 (必填)

type: 关节的类型 (必填)



类型	分类	特征	应用实例	是否需要 <limit>	是否需要 <axis>
revolute	转动类	绕指定轴线旋转，旋转角度有限制	门轴、机械臂、手指	需要限制角度范围	需要指定旋转轴
continuous	转动类	绕指定轴线无限旋转	风扇、轮子、电机轴	不需要限制角度范围	需要指定旋转轴
prismatic	平移类	沿指定轴线平移，移动距离有限制	滑轨、升降平台	需要限制平移范围	需要指定平移轴
planar	平移类	在垂直于轴线的平面内移动，可旋转	移动底座、悬浮平台	默认自由 (3DoF)	指定平面法向轴
fixed	限定类	完全固定，不允许任何运动	固定摄像头、电池座	不需要	不需要
floating	限定类	在空间中完全自由移动 (6DoF)	空间站模块、浮动传感器	不需要	不需要



2.7 <joint> 子标签

基础子标签 (必选)

- <parent>指定关节连接的父连杆
 - link (父连杆名称)
- <child>: 指定关节连接的子连杆
 - link (子连杆名称)

```
<joint name="joint1" type="revolute">
  <parent link="link1"/>
  <child link="link2"/>
  <origin xyz="0 0 1" rpy="0 0 0"/>
  <!-- joint1的位置和方向 -->
  <axis xyz="0 0 1"/>
  <!-- joint1围绕z轴旋转 -->
  <limit lower="-1.57" upper="1.57" effort="100.0" velocity="0.5"/>
</joint>
```

核心子标签 (可选)

- <origin>: 定义子连杆相对于父连杆的坐标变换 (位置和方向)
 - 有xyz和rpy属性, 同上
- <axis>
 - 指定关节的运动轴 (对 revolute、continuous、prismatic 类型有效)
 - xyz (运动轴向量, 默认 1 0 0)
- <limit>: 指定关节的运动范围 (对 转动和平移类型有效)
 - lower: 下限 (弧度或米)
 - upper: 上限 (弧度或米)
 - effort: 最大力 (牛顿或牛顿·米)
 - velocity: 最大速度 (弧度/秒或米/秒)



2.7 <joint> 子标签

其他子标签 (可选)

- <calibration>: 指定关节位置的校准信息, 用于设定零位或参考位置

- rising: 运动增加方向触发校准的位置
- falling: 运动减少方向触发校准的位置

- <dynamics>: 定义关节的动态特性, 如阻尼和摩擦

- damping: 阻尼系数
- friction: 摩擦系数

- <mimic>: 使当前关节模拟另一关节的运动

- joint: 被模拟的关节名称
- multiplier: 运动比例系数
- offset: 位置偏移 (弧度)

- <safety_controller>: 定义关节的安全限制, 防止超出物理范围

- soft_lower_limit: 软下限
- soft_upper_limit: 软上限
- k_position: 位置控制增益
- k_velocity: 速度控制增益

```
<!--被模拟关节-->
<joint name="main_joint" type="revolute">
  <parent link="base_link"/>
  <child link="arm_link"/>
  <origin xyz="0 0 0.5" rpy="0 0 0"/>
  <axis xyz="0 0 1"/>
  <limit lower="-1.57" upper="1.57" effort="100.0" velocity="0.5"/>
  <dynamics damping="0.5" friction="0.2"/>
  <safety_controller k_position="0.5" k_velocity="0.5" soft_lower_limit="-1.5" soft_upper_limit="1.5"/>
</joint>

<!--模拟关节-->
<joint name="aux_joint" type="revolute">
  <parent link="arm_link"/>
  <child link="forearm_link"/>
  <origin xyz="0 0 1.0" rpy="0 0 0"/>
  <axis xyz="0 0 1"/>
  <!--multiplier="0.5" offset="0.1", 代表aux的关节运动速度是main的一半, 且总是比main多一个固定偏移角度0.1rad-->
  <mimic joint="main_joint" multiplier="0.5" offset="0.1"/>
</joint>
```




案例：<joint>定义方法：以ugv rover为例

下面为左前轮joint的定义

```
<joint  
  name="left_up_wheel_link_joint"  
  type="continuous">  
  <origin  
    xyz="0.0855 0.08726 -0.041212"  
    rpy="1.5708 0 0" />  
  <parent  
    link="base_link" />  
  <child  
    link="left_up_wheel_link" />  
  <axis  
    xyz="0 0 -1" />  
</joint>
```

转动类，绕指定轴线无限旋转

关节坐标系相对于父link (base_link) 坐标系的平移

指定关节的父link为base_link

指定关节的子link为left_up_wheel_link

指定关节旋转的轴为(0, 0, -1)



2.8 基于Xacro的URDF编程优化

思考几个问题：

问题一：机器人硬件组成的复杂性

例如，对于一个人形机器人，有躯干、四肢、头，其下还有如手掌、脚掌等各部分零件，在编写URDF文件时，我们需要将所有部分都包含在一个<robot>根标签内吗，几百上千行在所难免！

问题二：机器人硬件组成的重复性

例如，对于一个轮式机器人底盘，四个轮子除了位置不同，其他属性具有高度一致性，更不用提履带轮了

问题三：参数变化带来的繁琐计算

例如，在设计多关节臂时，当某个关节的位置参数发生变动时，其他关节位置也要发生变动，依赖人工繁琐计算的话，失误率较高，但显然，这些相对位置关系的计算是有固定公式的

从这三类问题我们可以看出，在编写URDF的过程中，有许多工作是可以进一步结合**自动化编程**，引入参变量函数进行模块化封装，进而提高效率的。



2.8 基于Xacro的URDF编程优化

基于上述需求，ROS 2 引入 Xacro 优化 URDF 编程

Xacro: XML Macros 的缩写，提供宏功能，简化 URDF 文件的复杂性和重复性

- **Macros 宏：**一种自动化脚本，用于重复任务、增强代码重用性、简化复杂配置
- **常见应用：**
 - 微软 Office 宏：如 Word、Excel，录制操作（如格式化文本）为单一命令
 - 示例：Word 中录制宏，自动删除“图”与图号间的空格并加空格
 - 编译器预处理器宏：C/C++ 中定义代码片段或常量
 - 游戏宏：一键执行复杂操作（如连点、组合招式）

```
Sub InsertCaption()  
    '修改系统插入“题注”命令  
    '功能：自动删除标签与编号间的空格（英文除外），并在题注数字后添加一个空格；  
    'Dia Lab As String, startPt As Long, endPt As Long, anyrang As Range  
    'On Error Resume Next '发生错误时让程序继续执行下一句代码  
    'Application.ScreenUpdating = False '关闭屏幕更新，2013在此处关闭更新会导致输入框灰色不可选，故修正在弹出对话框之后  
  
    startPt = Selection.Start 'startPt标注起始点  
  
    '***将if条件隐藏起来即可实现——手动替换题注空格***  
    If Application.Dialogs(357).Show = 1 Then '插入“题注”对话框出来，如果按确定结束时执行以下程序，避免按取消后的空格，357可换成wdDialogInsertCaption  
        Application.ScreenUpdating = False '关闭屏幕更新  
  
        Lab = Dialogs(357).Label  
        endPt = Selection.Start 'endPt标注插入的题注部分终点  
        Selection.Start = startPt '选定插入的整个题注  
  
        '删除标签与编号间的空格（英文后的保留）  
        With Selection.Find  
            .Text = Lab &  
            .Forward = True 'False=向上查找，(True=向下查找)  
            .MatchWildcards = False '不使用通配符  
            If Lab Like "[0-9a-zA-Z]" Then '此处判断标签的最后一个字符是否为英文或数字，是则不删除空格  
            Else  
                .Replacement.Text = Lab  
                .Execute Replace:=wdReplaceOne '替换找到的第一个，此处用作删除空格  
                endPt = endPt - 1 '删除空格后，末位减1  
                Selection.End = endPt  
            End If  
        End With  
  
        '在题注数字后添加一个空格  
        Selection.Fields.ToggleShowCodes '切换域代码，这样才能用'd查找域  
        With Selection.Find  
            .Text = "d"  
            .Replacement.Text = "d "  
            .Forward = False 'False=向上查找，(True=向下查找)  
            .MatchWildcards = False '不使用通配符  
            .Execute Replace:=wdReplaceOne '替换找到的第一个，此处用作添加空格  
        End With  
  
        '选定整个插入的题注内容，将域代码切换回来  
        endPt = endPt + 1 '增加空格后，末位加1  
        With Selection  
            .Start = startPt  
            .End = endPt  
            .Fields.ToggleShowCodes '切换域代码（切换回来）  
        End With  
  
        '将光标定位至题注所在段落处  
        Selection.MoveRight Unit:=wdCharacter, Count:=1 '此句光标返回题注前的原始位置，对于已经输入标题的情况并不合适  
        '选择段尾回车符  
        With Selection.Find  
            .Text = "13"  
            .Forward = True 'False=向上查找，(True=向下查找)  
            .MatchWildcards = False '不使用通配符  
            .Wrap = wdFindContinue '继续查找  
            .Execute  
        End With  
        Selection.MoveLeft Unit:=wdCharacter, Count:=1 '定位到段尾回车前  
  
    End If  
    Application.ScreenUpdating = True '恢复屏幕更新  
End Sub
```



2.8 基于Xacro的URDF编程优化

使用步骤:

- 定义宏: 使用 `<xacro:macro>` 元素来定义一个宏, 指定宏的名称和接受的参数。
- 编写宏体: 在 `<xacro:macro>` 内部可以包含任何 URDF 或 Xacro 元素, 这些元素可以使用宏参数。
- 结束宏定义: 以 `</xacro:macro>` 结尾。
- 实例化宏: 通过 `<xacro:宏名>` 调用宏, 传入参数。

本质: Xacro 文件是 URDF 文件的扩展, 需通过 xacro 工具解析为标准 URDF

命名: 可命名为 xxx.xacro 或 xxx.urdf.xacro, ROS 2均可识别

```
<!--定义宏-->
<xacro:macro name="default_origin">
<origin xyz="0 0 0" rpy="0 0 0"/>
</xacro:macro>
<!--实例化调用-->
<xacro:default_origin />
<!-- 在这个例子中, 我们定义了一个<origin xyz="0 0 0" rpy="0 0 0"/>标签;
这样每次我们需要一个 原点在(0,0,0), 且没有rpy旋转的<origin>标签时, 就可以直接使用<xacro:default_origin />代替 -->
```



2.9 Xacro使用及语法

1.常量/变量定义

解决问题：参数复用及简单计算

功能：通过定义常量或变量，避免重复硬编码，支持参数化计算。

语法： `<xacro:property name="" value="" />`

示例：

- 定义全局变量，类似于 C 语言宏定义

```
<xacro:property name="WEIGHT" value="10.0"/>
<!-- 常见的还有-->
<xacro:property name="PI" value="3.14159265" />
```

- 使用 `${变量名}` 调用变量

```
<!-- 利用宏定义实现便捷的命名前缀: my_joint1 -->
<xacro:property name="prefix" value="my_" />
.....
<joint name="${prefix}joint1" type="revolute">
<!-- 关节的其他定义 -->
</joint>
```




2.9 Xacro使用及语法

2.<robot> 标签与文件调用

解决问题：模块化设计与文件整合

功能：通过 <xacro:include> 将多个 Xacro 文件组合，实现模块化设计，降低单一 URDF 文件的复杂性

场景：假设有一个机器人，分为基座(base)、手臂(arm)、头(head)三个部分，分别编写 base.xacro、arm.xacro、head.xacro，现在，需要在一个更大的主文件 `robot.urdf.xacro` 中组合起来

```
<!--base.xacros file-->
<robot name="my_base" xmlns:xacro="https://wiki.ros.org/xacro">
  <!-- 基座的定义 -->
</robot>
```

```
<!--arm.xacros file-->
<robot xmlns:xacro="https://wiki.ros.org/xacro">
  <!-- 手臂的定义 -->
</robot>
```

```
<!--head.xacros file-->
<robot xmlns:xacro="https://wiki.ros.org/xacro">
  <!-- 头的定义 -->
</robot>
```

```
<!--robot.urdf.xacro file-->
<robot name="my_robot" xmlns:xacro="https://wiki.ros.org/xacro">
  <xacro:include filename="$(find my_robot_package)/urdf/base.xacro"/>
  <xacro:include filename="$(find my_robot_package)/urdf/arm.xacro"/>
  <xacro:include filename="$(find my_robot_package)/urdf/head.xacro"/>
</robot>
```



2.9 Xacro使用及语法

3.带参宏

解决问题：参数化重复结构

功能：通过宏定义可复用模板，传入参数生成不同模型元素，减少重复代码。

语法：单参数宏

```
<xacro:macro name="宏名" params="参数名">
  <!-- 使用 ${参数名} -->
</xacro:macro>
<xacro:宏名 参数名="值" />
```

```
<xacro:macro name="macro_name"
params="params_name">
<xacro:macro_name params_name="" />
```

示例：

```
<!--定义-->
<xacro:macro name="default_link" params="prefix">
<link name="${prefix}_link1" />
</xacro:macro>
<!--实例化-->
<xacro:default_link prefix="my" />
<!-- 上述调用, 等价于<link name="my_link1" /> -->
```



2.9 Xacro使用及语法

3.带参宏

解决问题：参数化重复结构

功能：通过宏定义可复用模板，传入参数生成不同模型元素，减少重复代码。

语法：多参数与 XML 块

```
<xacro:macro name="宏名" params="参数1 参数2 *块参数">
  <tag>
    <xacro:insert_block name="块参数" />
  </tag>
</xacro:macro>
<xacro:宏名 参数1="值1" 参数2="值2">
  <!-- XML 块内容 -->
</xacro:宏名>
```

```
<xacro:macro name="macro_name" params="param1 param2 ... *paramX">
<!-- 在这里, *paramX 是特殊参数 (XML块参数) , 用星号`*`标识-->
  <tag>
    <xacro:insert_block name="paramX" />
  </tag>
</xacro:macro>
<!-- 在宏体内部, 定义出了一个元素 (tag, 例如link、joint...)
然后使用<xacro:insert_block name="paramX" />,
插入了一个名为paramX的XML块 (类似于占位符、回调函数) -->
```

示例：

```
<!--定义-->
<xacro:macro name="default_link" params="prefix *origin">
<link name="${prefix}_link1">
<xacro:insert_block name="origin" />
</link>
</xacro:macro>

<!--实例化-->
<xacro:default_link prefix="my">
<origin xyz="0 0 0" rpy="0 0 0" />
</xacro:default_link>

<!--上述动作等效于: -->
<link name="my_link1">
  <origin xyz="0 0 0" rpy="0 0 0" />
</link>
```



2.10 基于Solidworks的机器人建模

■ 现实问题：建模的复杂性

- 在前面的内容中，我们学习了如何直接编写urdf文件来创建机器人及其零部件。
不难看出，这样的方式在面对复杂且部件众多机器人时，将带来很庞大的工作量，并花费相当多的时间不断调整设计。
- 比如link和joint中的origin坐标参数，需要大量的手动调整工作

■ 解决方法：使用三维图形设计软件（CAD）来更快速、直观地设计机器人外观及构造。

- 使用主流的软件插件将三维模型导出为URDF：
 - **Fusion2URDF**：用于Autodesk Fusion 360的插件，可以直接将Fusion 360中设计的模型转换为URDF文件。
 - **SolidWorks to URDF Exporter**：专门用于将SolidWorks模型导出为URDF格式。这个插件直接集成在SolidWorks中，提供了一个用户友好的界面来帮助指定链接和关节的属性。
 - **Onshape to Robot**：将Onshape CAD模型转换为URDF文件的工具。Onshape是一个基于云的CAD系统。
 - **Blender to URDF**：Blender（一个开源的3D创作套件）可以用来编辑或创建模型，并通过脚本将这些模型转换为URDF格式。



2.10 基于Solidworks的机器人建模

■ SolidWorks: CAD及CAE软件, Dassault Systèmes开发, 1995年至今。

■ 广泛应用于各个行业, 包括汽车、航空航天、机械制造、电子产品、医疗设备、能源以及消费品设计等等。

■ 主要特性:

■ 优秀的GUI用户界面

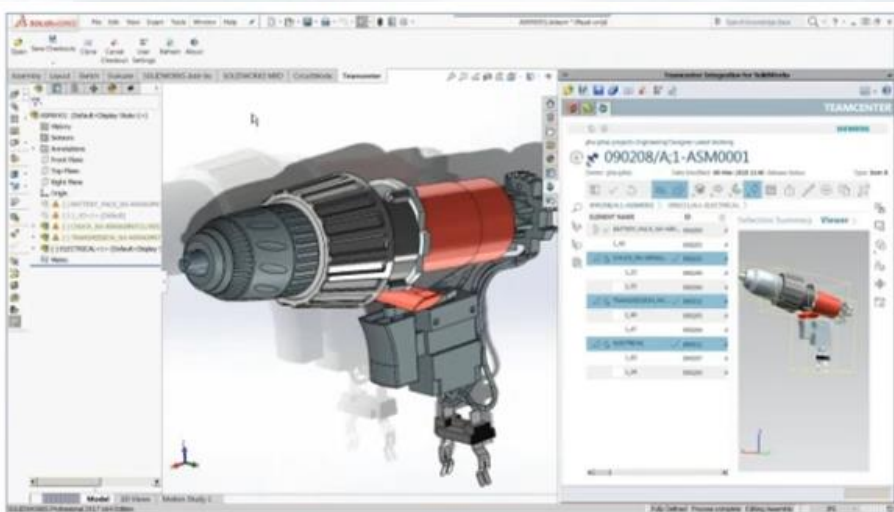
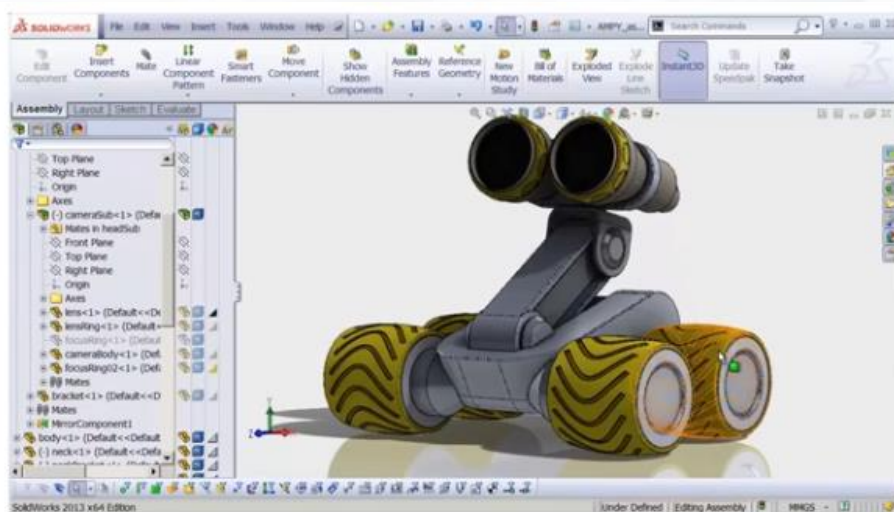
■ 3D设计: 支持3D实体建模和装配, 能够构建复杂的零件和装配体

■ 2D绘图和文档: 能生成详细的2D工程图和技术文档

■ 参数化和特征驱动建模: 支持参数化设计方法、以及重用和修改设计中的特定元素

■ 仿真系统: 内置的仿真工具允许用户在设计阶段进行强度、热分析、流体动力学分析等, 帮助识别潜在的设计问题; 同时也包括机械运动仿真。

■ 兼容性和集成: 支持主流三维模型格式, 能与其他CAD软件较好兼容





2.10 基于Solidworks的机器人建模

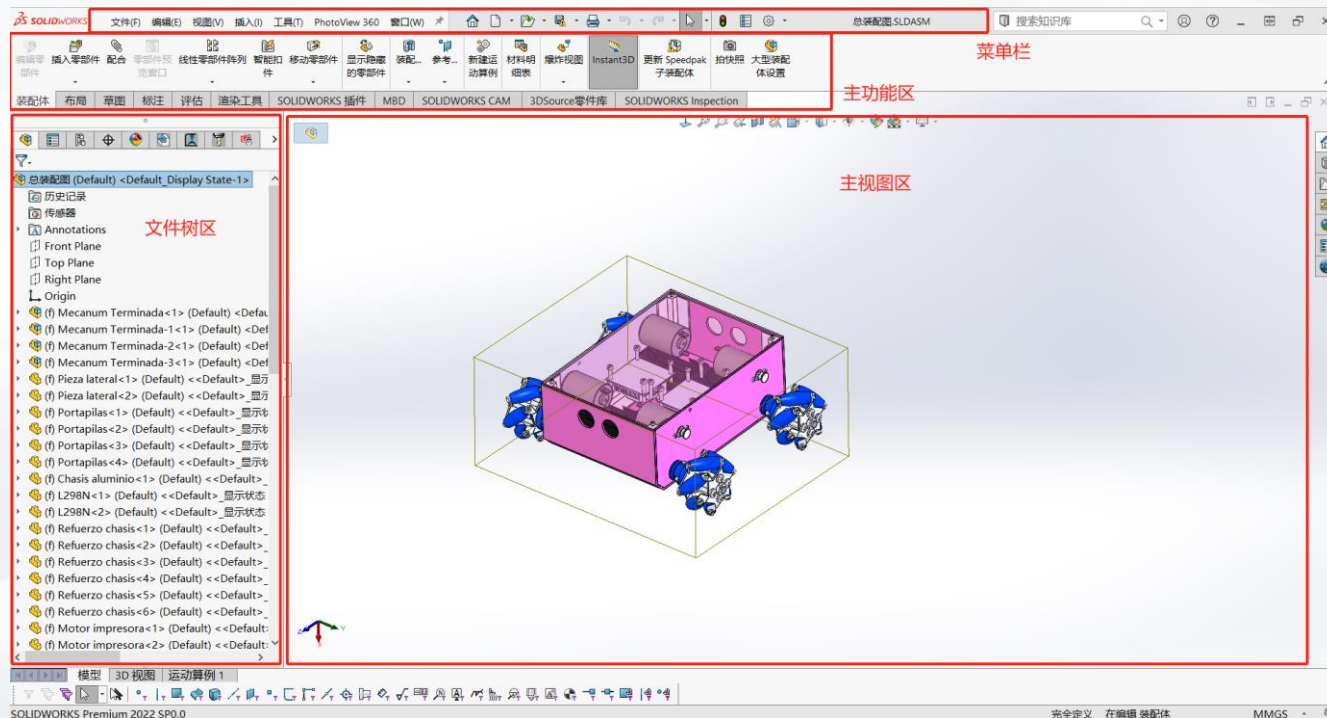
■ SolidWorks

■ 新建文件

- SolidWorks常用的两种文件是.SLDPART (part零件) 和.SLDASM (assembly装配体)

■ 主界面

- 主要有菜单栏、主功能区（草图绘制、三维操作；零部件 拼装等）、文件树区、主视图区



2.10 基于Solidworks的机器人建模

■ SolidWorks to URDF Exporter

- 下载地址：
 - solidworks: 微信号“火耳软件管家” 搜索下载
- 插件下载地址: https://wiki.ros.org/sw_urdf_exporter
 - SW2022版及以后的, 选用v1.6.1 (SolidWorks 2021); 避免使用SW2018
 - 下载完成后, 菜单栏--> 选项(齿轮图标)下拉 --> 插件--> SW2URDF, 确认勾选添加
 - 导出: 工具--> Tools(低版本为files) --> export as URDF



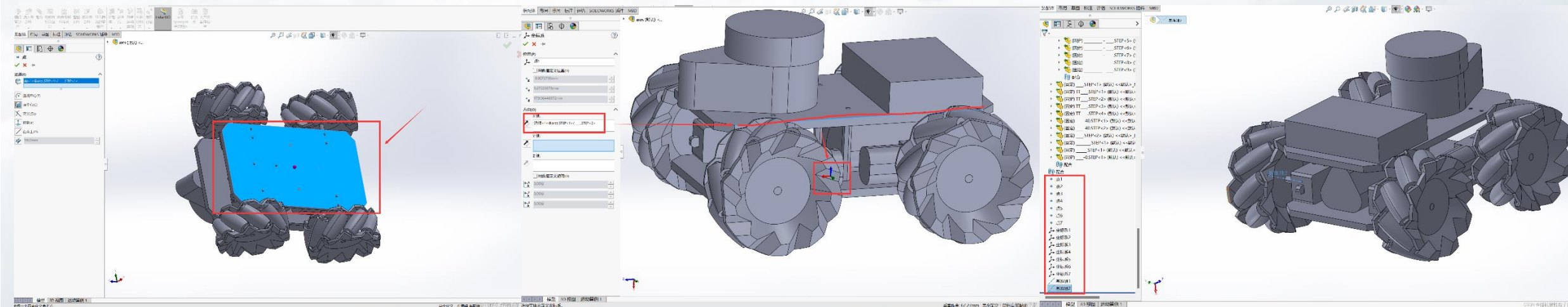
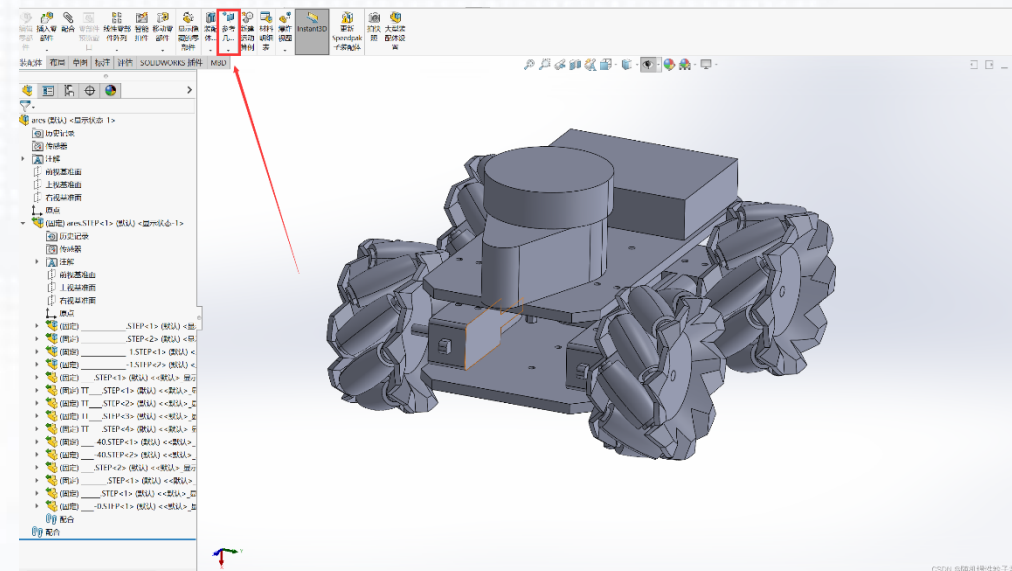
2.10 基于Solidworks的机器人建模

■ SolidWorks to URDF Exporter

■ 使用步骤:

■ 配置机器人的核心坐标系以及旋转轴

1. 参考几何体 ——> 点
2. 参考几何体 ——> 坐标轴
3. 参考几何体 ——> 基准轴



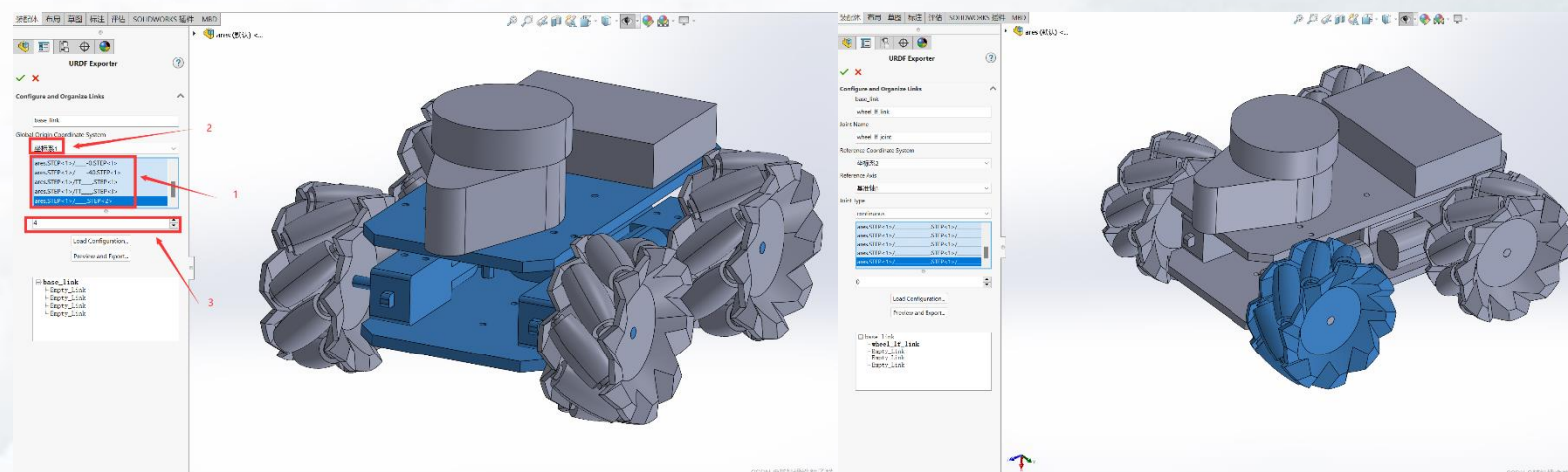
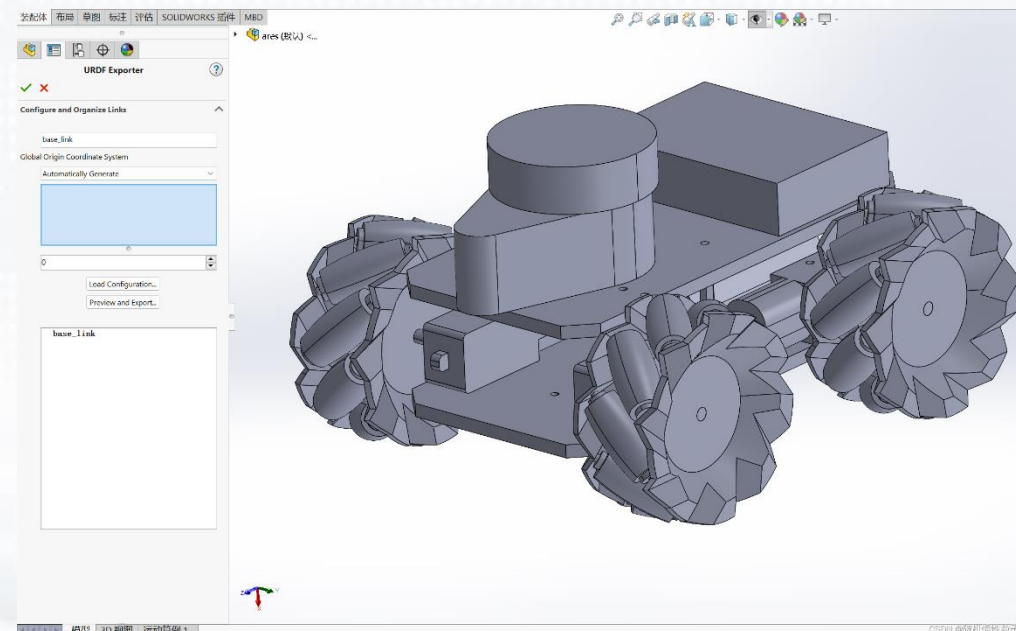
2.10 基于Solidworks的机器人建模

■ SolidWorks to URDF Exporter

■ 使用步骤:

■ Export as URDF

1. 选择零件
2. 选择坐标系
3. 设置child_link的数量
4. 配置child_link
 1. 选择零件
 2. 设置参考坐标系、基准轴
 3. 关节类型
 4. 设置child_link



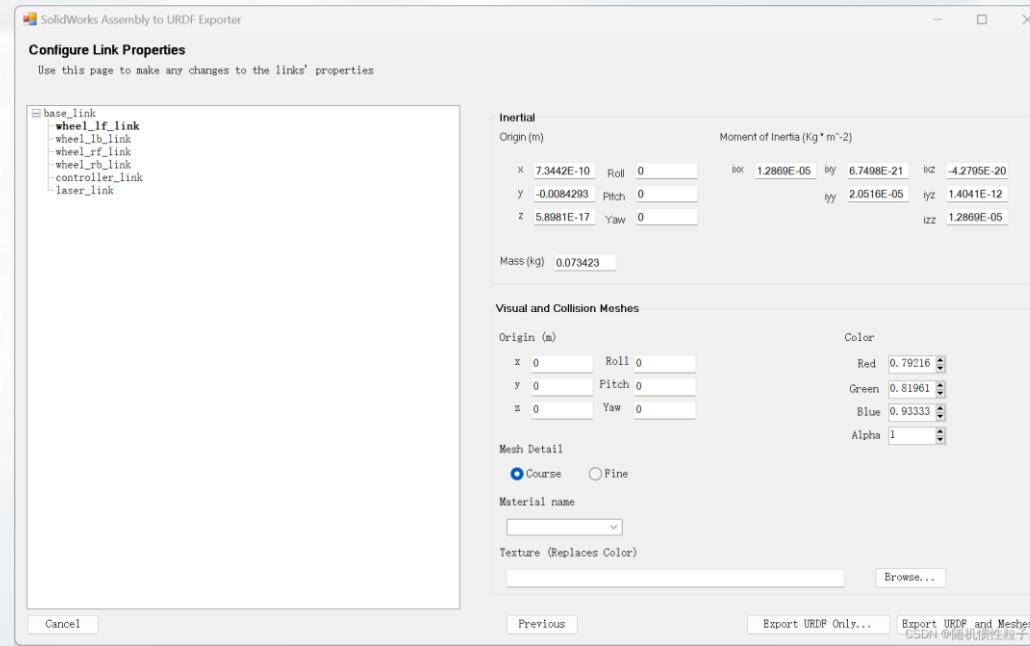
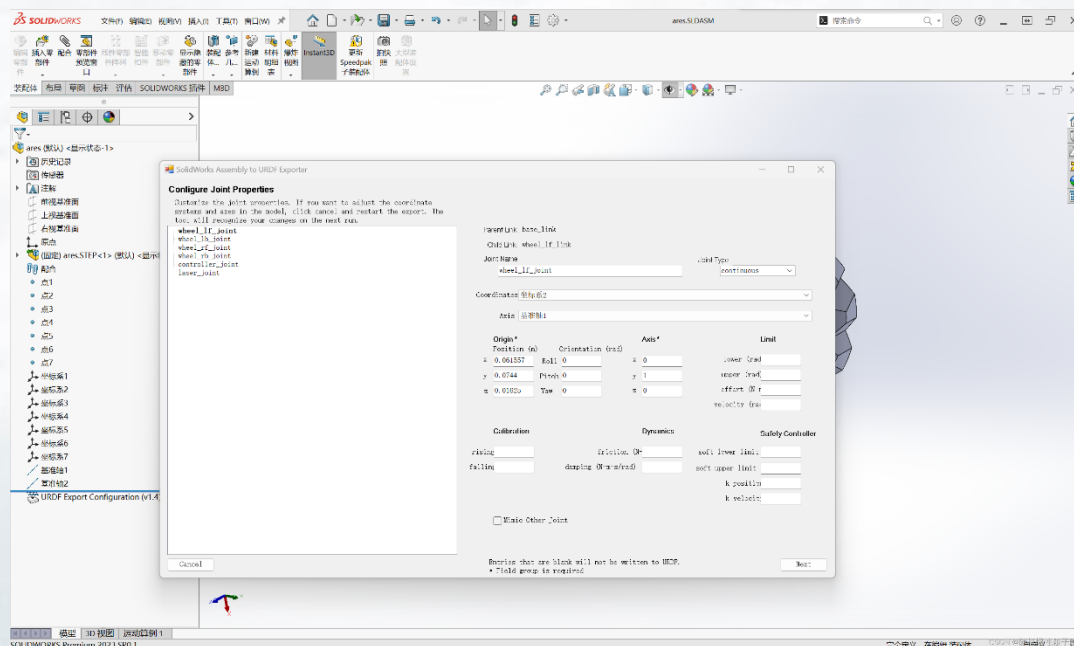
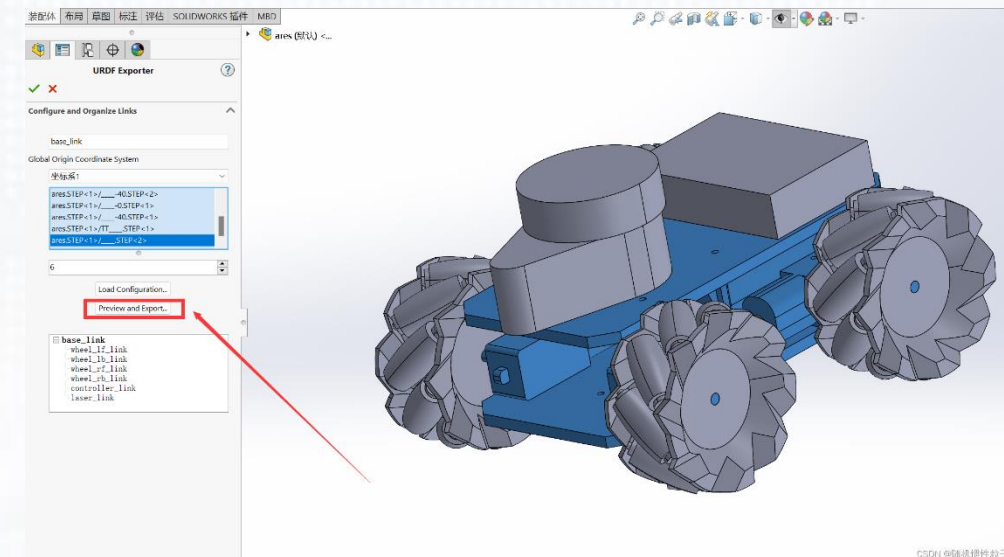
2.10 基于Solidworks的机器人建模

■ SolidWorks to URDF Exporter

■ 使用步骤:

■ Preview and Export

- 配置所有的joint
- 配置所有的link





机器人建模案例



同济大学
TONGJI UNIVERSITY



土木工程学院
COLLEGE OF CIVIL ENGINEERING



Gazebo安装

Gazebo版本与ROS2的版本对应

ROS2 Jazzy-Gazebo harmonic

- `sudo apt-get install lsb-release gnupg`
- `sudo curl https://packages.osrfoundation.org/gazebo.gpg --output /usr/share/keyrings/pkgs-osrf-archive-keyring.gpg`
- `echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/pkgs-osrf-archive-keyring.gpg]
http://packages.osrfoundation.org/gazebo/ubuntu-stable $(lsb_release -cs) main" | sudo tee
/etc/apt/sources.list.d/gazebo-stable.list > /dev/null`
- `sudo apt-get update`
- `sudo apt-get install gz-harmonic`
- 启动
- `gz sim`



机器人建模案例

配置rviz2——预设置rviz2的配置，无需启动rviz2后手动配置

```
4月21日 11:32
view_description.rviz2 - ugv_ws - Visual Studio Code

文件 编辑 选择 查看 转到 运行 终端 帮助

资源管理器
UGV_WS
  build
  images
  install
  log
  src
    ugv_else
    ugv_main
      ugv_base_node
      ugv_bringup
      ugv_chat_ai
      ugv_description
      launch
        display.launch.py
        ugv.launch.py
      meshes
      rviz
        view_descripti...
      urdf
      CMakeLists.txt
      package.xml
      ugv_gazebo
      ugv_interface
      ugv_nav
      ugv_slam
      ugv_tools
      ugv_vision
      ugv_web_app
  大纲
  时间线

src > ugv_main > ugv_description > rviz > view_description.rviz
1 Panels:
2   - Class: rviz_common/Displays
3     Help Height: 78
4     Name: Displays
5     Property Tree Widget:
6       Expanded:
7         - /Global Options1
8         - /Status1
9       Splitter Ratio: 0.5
10      Tree Height: 555
11   - Class: rviz_common/Selection
12     Name: Selection
13   - Class: rviz_common/Tool Properties
14     Expanded:
15       - /2D Goal Pose1
16       - /Publish Point1
17     Name: Tool Properties
18     Splitter Ratio: 0.5886790156364441
19   - Class: rviz_common/Views
20     Expanded:
21       - /Current View1
22     Name: Views
23     Splitter Ratio: 0.5
24   - Class: rviz_common/Time
25     Experimental: false
26     Name: Time
27     SyncMode: 0
28     SyncSource: ""
29 Visualization Manager:
30   Class: ""
31   Displays:
32     - Alpha: 0.5
33     Cell Size: 1
```



机器人建模案例

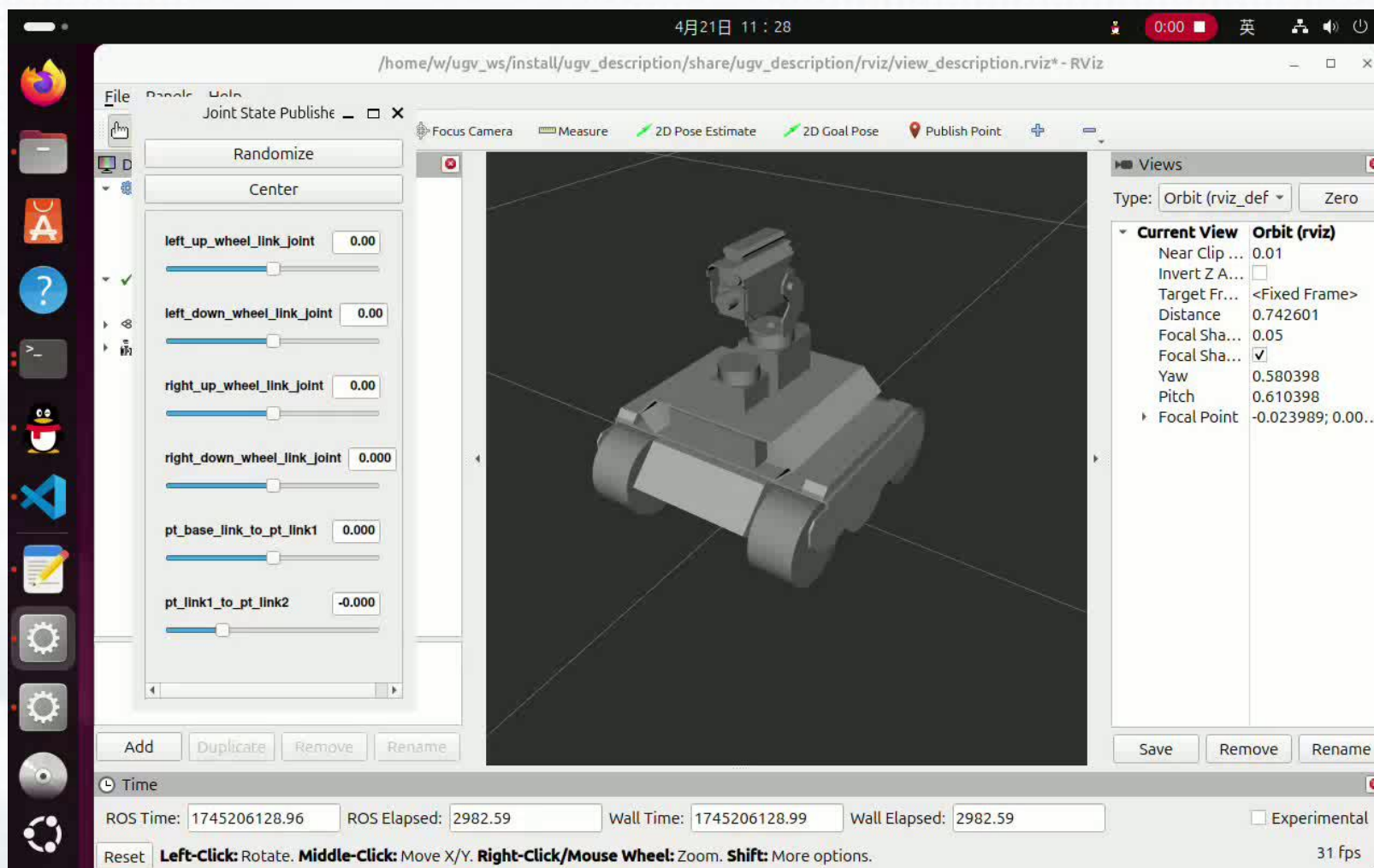
配置launch文件——选定rviz2配置文件、urdf启动文件

```
src > ugv_main > ugv_description > launch > ugv.launch.py
1  import os
2  from launch import LaunchDescription
3  from launch_ros.actions import Node
4  from ament_index_python.packages import get_package_share_directory
5
6  def generate_launch_description():
7      # Get package directory
8      pkg_share = get_package_share_directory('ugv_description')
9      urdf_file = os.path.join(pkg_share, 'urdf', 'ugv_rover.urdf')
10     rviz_config_file = os.path.join(pkg_share, 'rviz', 'view_description.rviz')
11
12     return LaunchDescription([
13         # Publish URDF and TF
14         Node(
15             package='robot_state_publisher',
16             executable='robot_state_publisher',
17             name='robot_state_publisher',
18             output='screen',
19             parameters=[{'robot_description': open(urdf_file).read()}]
20         ),
21         # Publish joint states with GUI
22         Node(
23             package='joint_state_publisher_gui',
24             executable='joint_state_publisher_gui',
25             name='joint_state_publisher_gui',
26             output='screen'
27         ),
28         # Launch RViz2 with view_description.rviz
29         Node(
30             package='rviz2',
31             executable='rviz2',
32             name='rviz2',
33             output='screen',
```



机器人建模案例

编译-source-ros2 launch





同濟大學
TONGJI UNIVERSITY



土木工程學院
COLLEGE OF CIVIL ENGINEERING

第五章 ROS2仿真

机器人仿真

完

潘玥

2025年5月5日